

UNIVERSIDAD SAN PABLO - CEU

ESCUELA POLITÉCNICA SUPERIOR

GRADO EN INGENIERÍA DE SISTEMAS DE INFORMACIÓN



TRABAJO FIN DE GRADO

Dispositivo IoT para la detección de caídas

IoT Device for fall detection

Autor: Daniel Gómez Obratzsov

Tutor: Aikaterini Koutsou

Mayo 2026



UNIVERSIDAD SAN PABLO-CEU

ESCUELA POLITÉCNICA SUPERIOR

División de Ingeniería

Calificación del Trabajo Fin de Grado

Datos del alumno

NOMBRE:

Datos del Trabajo

TÍTULO DEL PROYECTO:

Tribunal calificador

PRESIDENTE:

FDO.:

SECRETARIO:

FDO.:

VOCAL:

FDO.:

Reunido este tribunal el ____/____/_____, acuerda otorgar al Trabajo Fin de Grado presentado por D./Dña. _____ la calificación de _____

Resumen

Este Trabajo de Fin de Grado presenta el diseño, desarrollo y validación de un prototipo IoT para la detección de caídas, formado por un dispositivo wearable y una aplicación móvil Android. El dispositivo integra un Arduino Nano ESP32, un sensor inercial BNO055, una batería Li-Po y un sistema de alimentación portátil. El firmware del microcontrolador obtiene datos de aceleración, velocidad angular, orientación y gravedad, y ejecuta localmente un algoritmo basado en umbrales, energía de movimiento, postura e inactividad para identificar posibles caídas.

La aplicación móvil actúa como interfaz de usuario y módulo de respuesta ante emergencias. Permite configurar la conexión WIFI del dispositivo, iniciar la monitorización, gestionar contactos de emergencia, recibir alertas en tiempo real mediante WebSocket y cancelar falsas alarmas. Si una caída confirmada no es cancelada, la aplicación obtiene la ubicación del teléfono, envía SMS a los contactos configurados y realiza una llamada telefónica.

El sistema fue validado mediante pruebas del algoritmo, pruebas automáticas instrumentadas de la aplicación y pruebas funcionales del flujo completo. Los resultados muestran la viabilidad del prototipo como solución portable, local y explicable para detección de caídas.

Palabras Clave

IoT, detección de caídas, wearable, Arduino Nano ESP32, BNO055, Android.

Abstract

This Thesis Project presents the design, development and validation of an IoT prototype for fall detection, composed of a wearable device and an Android mobile application. The device integrates an Arduino Nano ESP32, a BNO055 inertial sensor, a Li-Po battery and a portable power management system. The microcontroller firmware collects acceleration, angular velocity, orientation and gravity data, and locally executes a threshold-based algorithm using motion energy, posture and inactivity to identify possible falls.

The mobile application acts as both the user interface and the emergency response module. It allows the user to configure the device WIFI connection, start monitoring, manage emergency contacts, receive real-time alerts through WebSocket and cancel false alarms. If a confirmed fall is not cancelled, the application obtains the phone location, sends SMS messages to the configured contacts and places a phone call.

The system was validated through algorithm accuracy tests, Android instrumented tests and functional tests of the complete alert flow. The results show the feasibility of the prototype as a portable, local and explainable solution for fall detection.

Keywords

IoT, fall detection, wearable, Arduino Nano ESP32, BNO055, Android.

Índice de contenidos

Capítulo 1	Introducción.....	1
1.1	Contexto del TFG	1
1.2	Objetivos.....	2
Capítulo 2	Gestión del proyecto	4
2.1	Modelo de ciclo de vida.....	4
2.2	Planificación.....	4
2.3	Presupuesto	5
Capítulo 3	Análisis.....	6
3.1	Estado del arte.....	6
3.2	Análisis del domino.....	10
3.3	Especificación de requisitos	12
Capítulo 4	Arquitectura y Diseño.....	17
4.1	Arquitectura del sistema	17
4.2	Diseño del hardware.....	21
4.2.1	Decisiones técnicas.....	21
4.2.2	Diseño de circuitos	23
4.3	Diseño del software	26
4.3.1	Decisiones técnicas.....	26
4.3.2	Diseño del algoritmo	28
Capítulo 5	Implementación, despliegue y validación	34
5.1	Detalles de implementación.....	34
5.1.1	Clases de la Aplicación Movil.....	34
5.1.2	Aprovisionamiento de red	37
5.1.3	Flujo de Monitorización.....	38
5.1.4	Flujo de detección de caídas	40
5.1.5	Funcionamiento del algoritmo	42
5.2	Plan de pruebas	44
5.2.1	Pruebas de precisión del algoritmo.....	44
5.2.2	Pruebas de usabilidad.....	47

Capítulo 6	Conclusiones y líneas futuras	49
6.1	Conclusiones	49
6.2	Líneas futuras	50
Bibliografía.....		51
Uso de herramientas de inteligencia artificial generativa		59
Anexo I – Librerías del proyecto		60
Anexo II – Diseño de la caja		64
Anexo III – Conexiones del hardware		68
Anexo IV – Dispositivo wearable		71
Anexo V – Wireframes y reultados.....		72
Anexo VI – Repositorio del proyecto		75
Anexo VII – Justificación de Requisitos.....		76

Índice de figuras

Ilustración 1. Diagrama de Gantt del proyecto	5	
Ilustración 2. Diagrama de Casos de Uso (Sistema general)	10	
Ilustración 3Diagrama de Casos de Uso (Setup).....	11	
Ilustración 4. Diagrama de Casos de Uso (Monitorización)	12	
Ilustración 5. Arquitectura del Sistema	17	
Ilustración 6. Arquitectura Lógica.....	18	
Ilustración 7. Arquitectura de Comunicación	20	
Ilustración 8. Esquemático de circuitos (primera versión)	24	
Ilustración 9. Esquemático de circuitos (versión final).....	25	
Ilustración 10. Circuito del PCB	26	
Ilustración 11. Diagrama State Machine del algoritmo.....	32	
Ilustración 12. Class Diagram.....	34	
Ilustración 13. Sequence Diagram - Aprovisionamiento WIFI.....	37	
Ilustración 14. Sequence Diagram - Monitorización (Conexión Arduino/Móvil)	39	
Ilustración 15. Sequence Diagram - Detección de Caídas	41	
Ilustración 16. Matriz de confusión.	45	
Ilustración 17. Detection Rate by Activity	46	
Ilustración 18. Graficas de caída real	Ilustración 19. Graficos de falso positivo ...	47
Ilustración 20. Esquema de caja interna.	64	
Ilustración 21. Esquema de caja externa.....	65	
Ilustración 22. Esquema de caja interna (diseño anterior).	66	
Ilustración 23. Comparacion de tamaño	67	

Ilustración 24. Conexión entre Arduino y sensor BNO055.....	68
Ilustración 25. Circuito de carga	69
Ilustración 26. Custom PCB.....	71
Ilustración 27. PCB dentro de la caja.....	71
Ilustración 28. Wireframes de la aplicación móvil	72
Ilustración 29. Pagina Principal.....	72
Ilustración 30. Página de setup	73
Ilustración 31. Página de Contactos	73
Ilustración 32. Página de añadir contacto	74

Índice de tablas

Tabla 1. Requisitos funcionales del sistema.....	15
Tabla 2. Requisitos no funcionales del sistema.....	16
Tabla 3. Resultados de la validación.....	45
Tabla 4. Librerías del Arduino	61
Tabla 5. Librerías de Android Studio	63
Tabla 6. Conexiones arduino-sensor	68
Tabla 7. Conexiones circuito carga	70

Capítulo 1

Introducción

1.1 Contexto del TFG

Las caídas son una de las principales causas de lesión entre personas mayores, y representan un problema importante tanto a nivel médico como social. Aunque muchas caídas pueden parecer accidentes puntuales, sus consecuencias pueden ser graves, especialmente cuando la persona afectada vive sola o no puede pedir ayuda después del incidente. Según la Organización Mundial de la Salud, se producen aproximadamente 684.000 muertes por caídas cada año, siendo la segunda causa mundial de muerte por lesiones no intencionadas. Además, las tasas de mortalidad más altas se concentran en adultos mayores de 60 años [1].

El riesgo de una caída no termina en el impacto inicial. En muchos casos, el mayor peligro aparece cuando la persona permanece en el suelo durante un tiempo prolongado sin recibir asistencia. La literatura médica asocia este tipo de situaciones con complicaciones como deshidratación, hipotermia, lesiones por presión, daño muscular y retraso en la atención médica, lo que puede empeorar considerablemente el pronóstico de la persona afectada [64].

Por este motivo, la detección temprana de caídas es un área de gran interés dentro de los sistemas de asistencia y monitorización. El objetivo de estos sistemas no es únicamente detectar que se ha producido una caída, sino reducir el tiempo entre el accidente y la respuesta de familiares, cuidadores o servicios de emergencia. En este contexto, los dispositivos portables y los sistemas IoT ofrecen una solución especialmente interesante, ya que permiten monitorizar el movimiento del usuario de

forma continua y enviar alertas de manera automática cuando se detecta una situación de riesgo [22][23][33].

Aun así, la detección de caídas sigue teniendo varios retos importantes. Una caída real puede compartir características con actividades normales del día a día, como sentarse rápido, agacharse, tumbarse o realizar un movimiento brusco. Por ello, uno de los mayores problemas de estos sistemas es diferenciar correctamente entre una caída verdadera y un falso positivo [16][27].

En este marco se sitúa el presente Trabajo de Fin de Grado, cuyo objetivo es diseñar y desarrollar un dispositivo IoT para la detección de caídas conectado a una aplicación móvil. El sistema estará formado por un sensor inercial, capaz de obtener información sobre aceleración, giro y orientación del usuario [2][46]. A partir de estos datos, el dispositivo analizará los patrones de movimiento para identificar posibles caídas y comunicar el evento a una aplicación móvil.

La aplicación móvil tendrá un papel central dentro del sistema, ya que actuará como interfaz entre el usuario, el dispositivo y los contactos de emergencia. Cuando el dispositivo detecte una posible caída, enviará una alerta a la aplicación, permitiendo avisar rápidamente a las personas configuradas por el usuario. Además, el proyecto plantea una arquitectura de comunicación flexible, utilizando WIFI en entornos domésticos para mejorar la cobertura [58].

1.2 Objetivos

El objetivo principal de este Trabajo de Fin de Grado es diseñar, desarrollar y validar un prototipo funcional de dispositivo IoT para la detección de caídas, conectado a una aplicación móvil Android. El sistema debe ser capaz de monitorizar el movimiento del usuario, identificar posibles caídas mediante un algoritmo y activar un flujo de alerta desde la aplicación móvil cuando se confirme una situación de riesgo. Para alcanzar este objetivo principal, se plantean los siguientes subobjetivos:

- **Diseño de un dispositivo wearable:** desarrollar un prototipo portable formado por un microcontrolador con un sensor inercial.
- **Desarrollo del firmware del dispositivo:** implementar en el microcontrolador la lectura periódica del sensor, el procesamiento de los datos, la ejecución del algoritmo de detección de caídas y la comunicación con la aplicación móvil.
- **Diseño del algoritmo de detección de caídas:** desarrollar un algoritmo de detección de caídas con el objetivo de diferenciar una caída real de otros movimientos bruscos.
- **Comunicación entre dispositivo y aplicación móvil:** implementar un sistema de comunicación fiable entre el dispositivo wearable y la aplicación móvil.
- **Desarrollo de la aplicación móvil Android:** crear una aplicación que actúe como interfaz de usuario, permita iniciar y detener la monitorización, gestionar contactos de emergencia, visualizar información del dispositivo y recibir alertas de caída.
- **Gestión de alertas y acciones de emergencia:** implementar un flujo de emergencia en la aplicación que se encargue de contactar a los contactos de emergencia cuando se detecte una caída.

Capítulo 2

Gestión del proyecto

2.1 Modelo de ciclo de vida

El desarrollo de este proyecto se ha llevado a cabo siguiendo un modelo de ciclo de vida iterativo e incremental, con un enfoque de las metodologías ágiles. Durante el desarrollo se realizaron Sprints bisemanales con la tutora del proyecto. En los cuales se revisaba las tareas en revisión, se elegían desde el backlog las tareas para el nuevo sprint y se realizaba una retrospectiva del sprint realizado. Este funcionamiento permitió adaptar la planificación a los problemas encontrados durante el desarrollo, especialmente en aspectos relacionados con la integración del hardware, la comunicación entre el dispositivo y la aplicación móvil, y el ajuste del algoritmo de detección de caídas.

2.2 Planificación

La planificación del proyecto fue para un periodo de 8 meses. La realización de las tareas de desarrollo del hardware y software siguieron la planificación, salvo la tarea de documentación del proyecto que se sufrió retrasos.

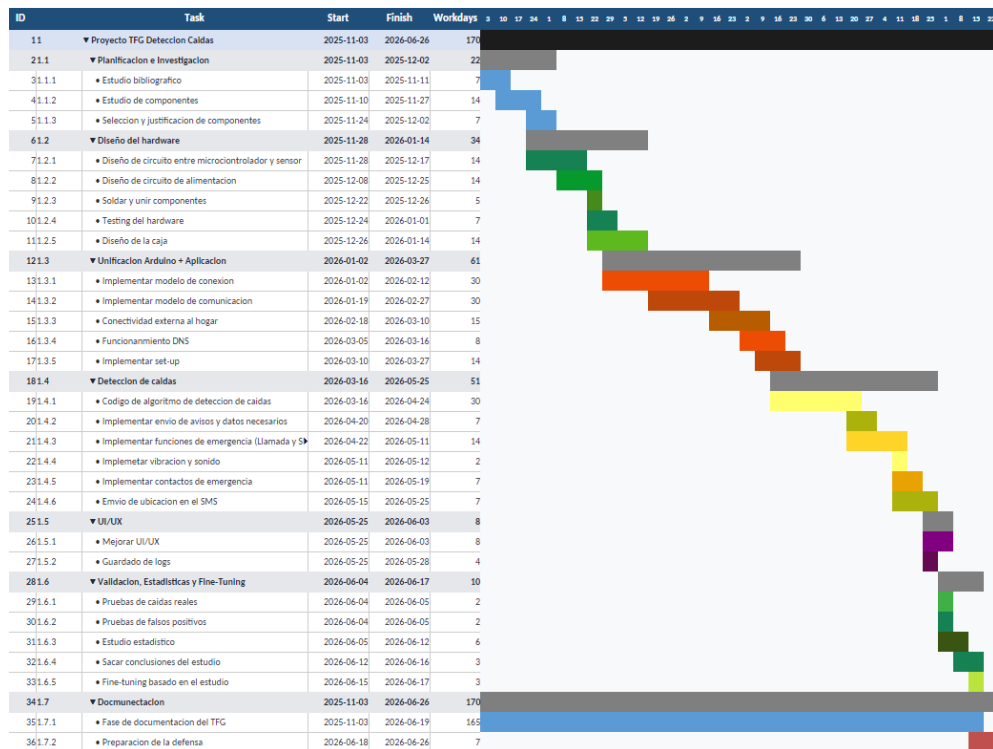


Ilustración 1. Diagrama de Gantt del proyecto

2.3 Presupuesto

El presupuesto del proyecto corresponde principalmente de componentes electrónicos, materiales y herramientas necesarias para hacer el prototipo. El coste total aproximado es de **229€**. En los componentes principales se incluye el Arduino Nano ESP32, con un coste de **25€**. El sensor BNO055, con un coste de **40€**. El módulo de carga y boost PowerBoost 500C, con un coste de **27 €**. Y la batería LiPo de 500mA a 3.7V, con un coste de **12 €**. También se adquirió una PCB personalizada, cuyo coste total aproximado de **45 €**. Además, se compraron componentes para el primer diseño: módulo genérico de carga y boost por **10 €**, placas adaptadoras de USB-C por **5 €** y placas adaptadoras de micro-USB por **5 €**. Por último, se incluyen materiales y herramientas de fabricación los cuales han costado aproximadamente **60 €**

Capítulo 3

Análisis

3.1 Estado del arte

La detección automática de caídas es un campo de investigación amplio dentro de los sistemas de asistencia, monitorización e IoT aplicados a la salud [22][33]. Las soluciones existentes pueden estudiarse desde tres perspectivas principales: el hardware utilizado para captar el evento, el software empleado para comunicar y gestionar la información, y el algoritmo encargado de diferenciar una caída real de una actividad cotidiana.

Hardware en sistemas de detección de caídas

Los sistemas de detección de caídas pueden clasificarse, de forma general, en tres grupos: sistemas basados en sensores ambientales, sistemas basados en visión artificial y sistemas basados en dispositivos wearables. Los sensores y los sistemas de visión permiten monitorizar un espacio sin que el usuario lleve un dispositivo encima, pero presentan limitaciones: dependen del entorno, no ofrecen cobertura fuera de la zona instalada y, en el caso de las cámaras, pueden plantear problemas de privacidad.

En soluciones de wearable, el usuario lleva un sensor en el cuerpo, normalmente integrado en un teléfono móvil, un reloj inteligente, o un dispositivo colocado en zonas como la cintura, el pecho o el tronco [15][28]. Los teléfonos móviles tienen la ventaja de incorporar acelerómetro, giroscopio, capacidad de procesamiento y conectividad, pero su posición no siempre es constante, el usuario puede llevarlos en distintas zonas o no llevarlos. Esta variabilidad dificulta interpretar los datos de movimiento de forma consistente [28][33]. Los relojes inteligentes y pulseras de actividad son una alternativa cómoda y aceptada por el usuario, ya que suelen llevarse de forma continua e incorporan sensores inerciales y comunicación inalámbrica. Sin embargo, al estar situados en la muñeca, pueden registrar movimientos bruscos del brazo que no representan necesariamente el movimiento global del

cuerpo, lo que puede aumentar la dificultad de diferenciar una caída real de actividades normales.

Otra línea habitual es el uso de dispositivos wearables específicos basados en unidades de medida inercial o IMU [28][31]. Estos dispositivos suelen integrar acelerómetros, giroscopios y, en algunos casos, magnetómetros. Su ventaja es que pueden colocarse en una posición más representativa del movimiento corporal, como la cintura o el tronco, y permiten diseñar el hardware según las necesidades concretas del sistema. En estudios de detección de caídas se ha observado que combinar aceleración, velocidad angular y postura puede mejorar la detección frente a métodos que utilizan únicamente aceleración [11][31].

En cuanto a la unidad de procesamiento, los sistemas wearables pueden utilizar desde microcontroladores sencillos hasta placas con mayor capacidad de cálculo. Para detección local basada en umbrales o reglas, un microcontrolador suele ser suficiente, siempre que disponga de capacidad para leer sensores, ejecutar el algoritmo y comunicarse de forma inalámbrica [34][46]. En cambio, soluciones basadas en visión artificial o aprendizaje profundo suelen requerir mayor capacidad de procesamiento o apoyo de un servidor externo.

Software y tecnologías de comunicación

Desde el punto de vista software, los sistemas de detección de caídas suelen incluir una aplicación móvil, un firmware embebido y un mecanismo de comunicación entre ambos. La aplicación móvil actúa como interfaz entre el usuario y el sistema, permitiendo configurar el dispositivo, visualizar información, gestionar contactos de emergencia y activar acciones como notificaciones, SMS o llamadas.

Para el desarrollo móvil existen varias alternativas. En Android, los lenguajes más habituales son Kotlin y Java, siendo Kotlin la opción moderna recomendada dentro del ecosistema Android [51][52]. En iOS, el desarrollo nativo se realiza principalmente con Swift. También existen tecnologías multiplataforma, como Flutter o React Native, que permiten desarrollar una única base de código para Android e iOS, aunque pueden añadir capas intermedias al acceso a funciones específicas del sistema operativo.

El desarrollo nativo presenta ventajas importantes, ya que permite una integración directa con APIs del sistema como servicios en primer plano, notificaciones, ubicación, vibración, llamadas telefónicas, SMS, almacenamiento local y comunicación de red [35][37]. Estas funciones son relevantes en un sistema de detección de caídas, donde la aplicación debe mantenerse activa durante la monitorización y reaccionar rápidamente ante una alerta.

Respecto a la comunicación entre el dispositivo wearable y la aplicación móvil, las tecnologías más habituales son Bluetooth/BLE, WIFI, WIFI Direct y comunicación mediante servidor o nube [22]. Bluetooth Low Energy es frecuente en wearables por su bajo consumo y porque permite comunicación directa entre sensor y teléfono, aunque depende de la proximidad entre ambos dispositivos. WIFI permite mayor cobertura dentro de un hogar y facilita la comunicación local cuando ambos dispositivos están conectados a la misma red. WIFI Direct permite comunicación directa sin router, pero aumenta la complejidad de configuración.

En sistemas IoT domésticos es común combinar varios mecanismos de comunicación. HTTP se utiliza para operaciones puntuales de configuración, como enviar credenciales de red o consultar el estado de un dispositivo. Para comunicación en tiempo real, WebSocket resulta adecuado porque mantiene una conexión persistente entre cliente y servidor, permitiendo que el dispositivo envíe eventos a la aplicación sin que esta tenga que realizar consultas periódicas [55][62]. Además, tecnologías de descubrimiento de servicios como mDNS o NSD facilitan que la aplicación encuentre automáticamente dispositivos disponibles en la red local [53].

Algoritmos de detección de caídas

Los algoritmos de detección de caídas basados en sensores inerciales pueden dividirse en varios grupos: algoritmos basados en umbrales, algoritmos basados en energía o ventanas temporales, algoritmos que combinan aceleración, giroscopio y postura, algoritmos de detección pre-impacto y algoritmos basados en aprendizaje automático [23][33].

Los algoritmos basados en umbrales son una de las aproximaciones más simples y frecuentes [24][25]. Normalmente calculan la magnitud de la aceleración y detectan una posible caída cuando esta supera un valor determinado. Su ventaja principal es que son fáciles de

implementar y tienen bajo coste computacional. Sin embargo, algunas actividades cotidianas pueden generar picos de aceleración elevados. Por este motivo, los algoritmos basados únicamente en aceleración pueden producir falsos positivos [16][23].

Para reducir esta limitación, varios trabajos combinan aceleración con velocidad angular y postura. En algunos estudios se propone utilizar giroscopios junto con información de postura derivada de acelerómetros, con el objetivo de reducir falsos positivos y falsos negativos frente a métodos basados únicamente en aceleración [11]. Esta idea es relevante porque una caída no es solo un impacto, sino también por una rotación del cuerpo y por una postura posterior diferente a la posición normal.

Otra línea de trabajo utiliza características de energía calculadas en ventanas temporales. En algunos estudios proponen una metodología de doble umbral basada en acelerómetro triaxial, utilizando características como la magnitud vectorial y el área de la señal [13]. Según los resultados publicados, el método alcanzó una precisión del 99 % en el conjunto Mobifall y del 97 % en un conjunto de datos propio [13]. Este tipo de enfoque es interesante porque no evalúa únicamente una muestra aislada, sino el comportamiento de la señal durante un intervalo de tiempo.

También se han propuesto algoritmos de detección pre-impacto, que utilizan una IMU y características como aceleración, velocidad angular e inclinación del tronco [12]. Este tipo de solución es útil cuando el objetivo es activar un mecanismo de protección antes de que el usuario golpee el suelo. No obstante, para este prototipo, resulta más importante confirmar correctamente el evento y reducir falsas alarmas.

Los algoritmos basados en aprendizaje automático representan otra línea relevante. En lugar de definir manualmente todos los umbrales, se entrenan clasificadores a partir de datos etiquetados con técnicas como árboles de decisión, SVM o redes neuronales. Estos métodos pueden alcanzar buenos resultados, pero requieren conjuntos de datos representativos, entrenamiento previo y validación rigurosa.

Un aspecto importante es la diferencia entre validar algoritmos con caídas simuladas y hacerlo con caídas reales [16][30]. Muchos estudios obtienen resultados altos en entornos

controlados, pero su rendimiento puede cambia cuando se prueban con usuarios reales, movimientos no previstos o situaciones cotidianas.

3.2 Análisis del domino

Los actores que participan en este proyecto son los siguientes:

- **Usuario:** La persona que lleva el dispositivo puesto y a la que se le quiere monitorizar.
- **Contacto de emergencia:** Personas a las que se les debe contactar por llamada y/o SMS cuando se haya detectado una caída.
- **Aplicación móvil:** La propia aplicación, la cual se encarga de llevar a cabo las acciones de emergencia una vez se haya detectado la caída. Reside en el teléfono móvil del usuario.
- **Dispositivo de detección de caídas:** Wearable llevado por el usuario el cual con sensores detecta una caída y se lo comunica a la aplicación móvil.
- **Red WIFI:** La red a la que se debe conectar el móvil donde reside la aplicación, y el dispositivo de detección de caídas.

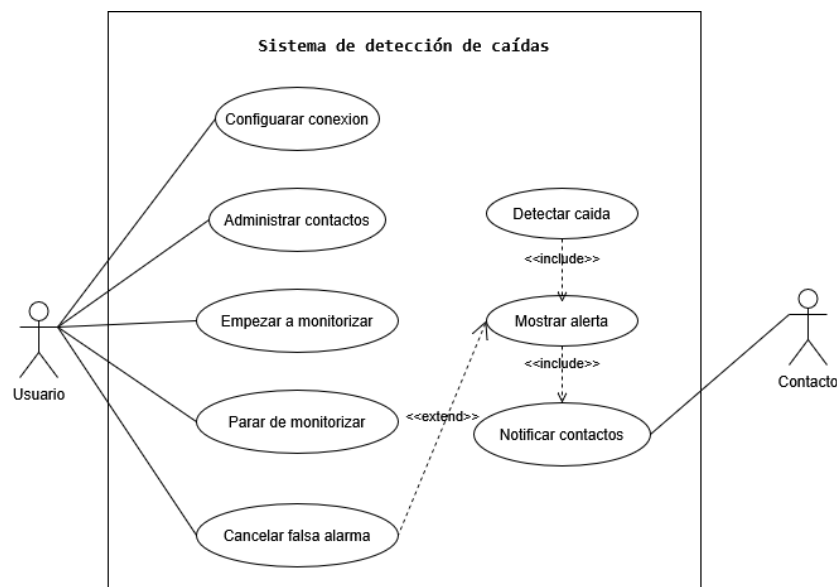


Ilustración 2. Diagrama de Casos de Uso (Sistema general)

La figura 2 son las funcionalidades principales de la aplicación desde el punto de vista del usuario. El usuario puede configurar la conexión con el dispositivo, administrar los contactos

de emergencia, empezar o parar la monitorización y cancelar una falsa alarma si el sistema detecta una caída incorrectamente. El sistema incluye el caso de uso de detectar caída, que hace que se muestre una alerta que, si esta no es cancelada, hace notificar a los contactos de emergencia. El contacto de emergencia es el actor externo porque no interactúa directamente con la aplicación, pero recibe las notificaciones de ella.

La figura 3 es el setup del dispositivo es el proceso inicial necesario para configurar la conectividad WIFI del Arduino. El actor principal es el usuario, que debe encender el dispositivo, conectar el teléfono móvil a la red temporal creada por el Arduino e introducir las credenciales de la red WIFI del hogar. Una vez introducidos los datos, se guarda la configuración en la memoria persistente del dispositivo y posteriormente intenta conectar el Arduino a la red WIFI indicada. La red WIFI actúa como actor externo al ser el medio al que el dispositivo debe conectarse para poder funcionar.

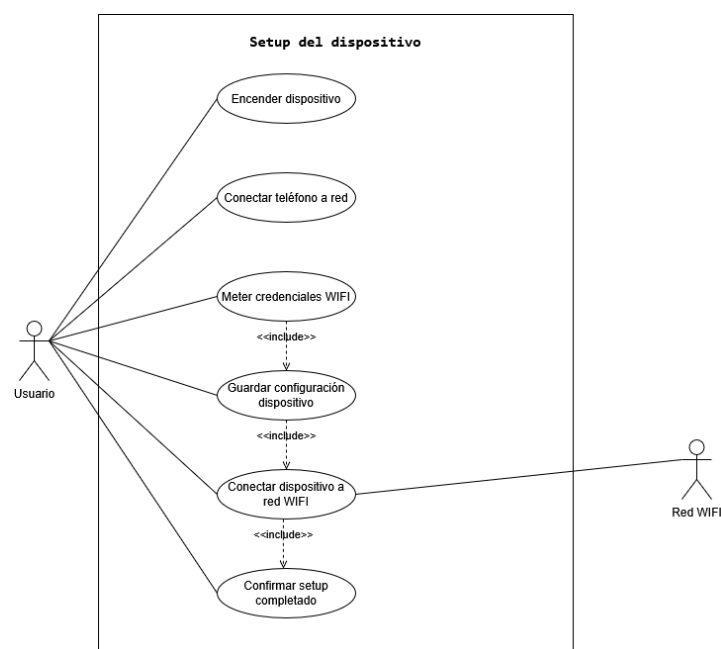


Ilustración 3 Diagrama de Casos de Uso (Setup)

La figura 4 es el funcionamiento normal del sistema una vez que el dispositivo ya ha sido configurado. El usuario inicia el proceso llevando puesto el wearable y activando la monitorización desde la aplicación móvil. Después el dispositivo lee los datos de movimiento

dados por el sensor, los analiza localmente y envía información a la aplicación. El wearable es el encargado de leer los datos, analizar el movimiento, detectar una posible caída, confirmar la caída y enviar la alerta a la app cuando se cumplen las condiciones necesarias. La aplicación móvil muestra los datos recibidos y presenta la alerta en caso de caída confirmada.

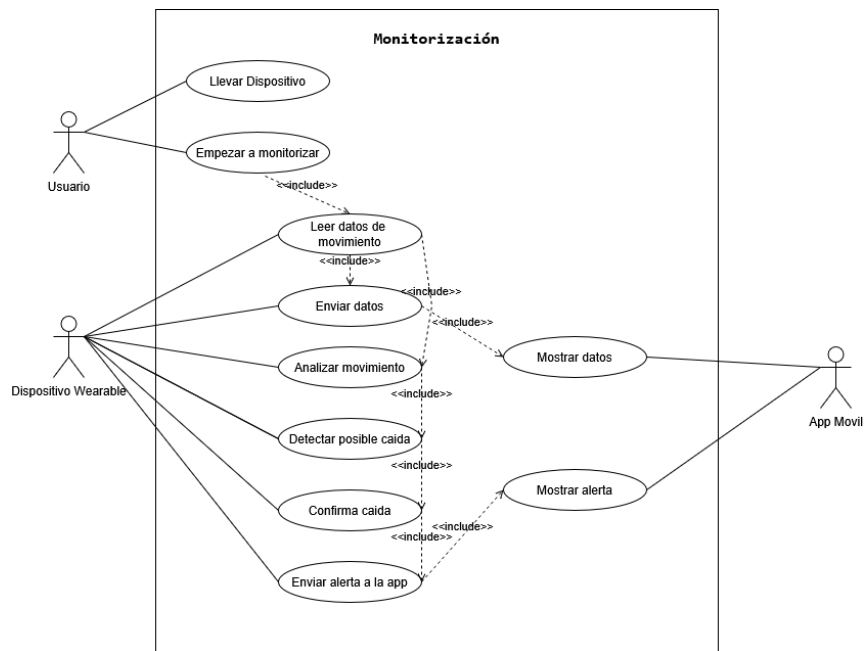


Ilustración 4. Diagrama de Casos de Uso (Monitorización)

3.3 Especificación de requisitos

Ref.	Requisito	Descripción
RF-1	Inicialización del dispositivo	El detector de caídas se debe inicializar, así encendiendo el Arduino Nano ESP32, el BNO055 y los servicios de comunicación necesarios.
RF-2	Activación del modo setup	Si el dispositivo no puede conectarse a una red WIFI guardada, debe entrar automáticamente en modo setup.
RF-3	Punto de acceso de setup	En modo setup, el dispositivo debe crear un punto de acceso WIFI.
RF-4	Aprovisionamiento de credenciales WIFI	La app móvil debe permitir al usuario introducir credenciales WIFI y enviarlas al dispositivo mediante POST /provision.

RF-5	Varias redes WIFI	El dispositivo debe almacenar hasta cinco redes WIFI, permitiendo que la primera red configurada tenga prioridad.
RF-6	Almacenamiento persistente de WIFI	El ESP32 debe almacenar las credenciales WIFI en preferencias no volátiles para que permanezcan disponibles tras reiniciar.
RF-7	Reinicio automático tras el aprovisionamiento	Después de un aprovisionamiento correcto, el ESP32 debe reiniciarse e intentar entrar en modo de monitorización normal.
RF-8	Conexión WIFI en modo normal	En modo normal, el ESP32 debe conectarse a una de las redes WIFI guardadas.
RF-9	Descubrimiento de servicio (DNS)	El ESP32 debe anunciarse en la red local usando el nombre de host, del servicio y el puerto 81.
RF-10	Conexión manual respaldo	La app debe permitir al usuario introducir manualmente un nombre de host o una dirección IP.
RF-11	Conexión websocket	La app debe establecer una conexión websocket con el ESP32 usando <code>ws://<host>:81/</code> .
RF-12	Iniciar monitorización	El usuario debe poder iniciar el servicio de monitorización desde la app.
RF-13	Detener monitorización	El usuario debe poder detener el servicio de monitorización desde la app.
RF-14	Adquisición de sensores	El ESP32 debe leer periódicamente del sensor BNO055 los datos de aceleración lineal, giroscopio, orientación Euler y vector gravedad.
RF-15	Cálculo de métricas de caída	El ESP32 debe calcular métricas relacionadas con la caída, como magnitud de aceleración, magnitud del giroscopio, energía de caída, energía total y ángulo de postura.
RF-16	Detección de caída posible	El ESP32 debe detectar una posible caída cuando el movimiento supere los umbrales configurados de impacto y velocidad angular.
RF-17	Captura de ventana de evento	Cuando se detecte una candidata a caída, el ESP32 debe capturar una ventana de evento con muestras previas y posteriores al trigger.
RF-18	Confirmación de caída	El ESP32 debe confirmar una caída solo cuando el evento candidato vaya seguido de condiciones de postura tumbada e inactividad durante el tiempo requerido.
RF-19	Envío de mensajes en tiempo real	El ESP32 debe enviar mensajes en tiempo real a la app usando tipos de mensaje websocket.

RF-20	Visualización en vivo	La app móvil debe mostrar en tiempo real los mensajes recibidos del Arduino+BNO055 (estado de conexión, debug y eventos de caída)
RF-21	Logs en local de datos	La app debe guardar en archivos de log en local los mensajes recibidos del Arduino y los eventos de la aplicación para estadísticas.
RF-22	Gestión de contactos de emergencia	La app debe permitir al usuario añadir, ver y eliminar contactos de emergencia con nombre y número de teléfono.
RF-23	Gestión de permisos	La app debe solicitar los permisos necesarios para SMS, llamadas telefónicas, notificaciones, ubicación, vibración y comunicación de red.
RF-24	Recepción de alerta de caída	Cuando la app reciba un mensaje de caída o un mensaje que contenga FALL DETECTED, debe iniciar el flujo de alerta por caída.
RF-25	Cuenta atrás de alerta	La app debe iniciar una cuenta atrás de 15 segundos antes de ejecutar las acciones de emergencia.
RF-26	Cancelación de falsa alarma	El usuario debe poder cancelar pulsando el botón de falsa alarma antes de que termine la cuenta atrás.
RF-27	Alerta sonora y vibración	Durante la cuenta atrás, la app debe alertar al usuario mediante vibración y sonido repetido.
RF-28	Obtención de ubicación de emergencia	La app debe obtener la ubicación actual o la última conocida del teléfono.
RF-29	Generación de enlace de Google Maps	La app debe generar un enlace de Google Maps usando la latitud y longitud disponibles.
RF-30	Envío de SMS de emergencia	Si la alerta no ha sido cancelada, la app debe enviar un SMS de emergencia a todos los contactos.
RF-31	Llamada telefónica de emergencia	Si la alerta no ha sido cancelada, la app debe llamar al primer contacto de emergencia guardado.
RF-32	Llamada en altavoz	Al hacer la llamada, se pone automáticamente en altavoz para la posibilidad de hablar aun estando lejos.
RF-33	Gestión de reconexión	Si la conexión websocket falla de forma inesperada, la app debe intentar reconectarse.
RF-34	Recuperación tras pérdida de WIFI	Si el Arduino pierde conexión WIFI, debe reiniciarse e intentar reconectarse usando las redes guardadas.

RF-35	Compartir registros	La app debe permitir al usuario o al evaluador compartir o exportar los logs generados..
--------------	---------------------	--

Tabla 1. Requisitos funcionales del sistema.

Ref.	Requisito	Descripción
RNF-01	Usabilidad para usuarios mayores	La app debe proporcionar una interfaz sencilla y fácil de entender, adecuada para usuarios mayores.
RNF-02	Precisión del algoritmo	Se deben detectar como mínimo un 95% de las caídas reales.
RNF-03	Respuesta rápida ante emergencias	El sistema debe minimizar el tiempo entre la confirmación de la caída y el inicio del flujo de respuesta de emergencia.
RNF-04	Capacidad de respuesta	El procesamiento de sensores y la comunicación de alertas deben ser lo bastante rápidos como para permitir una monitorización casi en tiempo real.
RNF-05	Fiabilidad	El sistema debe seguir monitorizando mientras el dispositivo y el teléfono estén encendidos y conectados.
RNF-06	Robustez de la conexión	El sistema debe admitir descubrimiento automático, conexión manual de respaldo y reconexión tras fallos de comunicación.
RNF-07	Reducción de falsos positivos	El algoritmo de detección de caídas debe reducir las falsas alarmas lo máximo posible.
RNF-08	Falsa Alarma	La app debe dar al usuario una breve oportunidad para cancelar falsas alarmas antes de contactar con los contactos de emergencia.
RNF-09	Disponibilidad en entorno doméstico	La comunicación WIFI debe proporcionar cobertura suficiente para un uso normal dentro de un entorno doméstico.
RNF-10	Portabilidad	El dispositivo wearable debe ser lo bastante compacto y cómodo para el uso diario.
RNF-11	Autonomía de batería	El dispositivo debe estar diseñado para funcionar con una batería portátil y consumir energía de forma eficiente para pruebas prácticas.
RNF-12	Seguridad — permisos	La app debe solicitar sólo los permisos necesarios para sus funciones de emergencia y debe dejar claro al usuario su propósito.
RNF-13	Persistencia de datos	Las credenciales WIFI, los contactos de emergencia y los registros deben seguir disponibles después de reiniciar la app o el dispositivo.

RNF-14	Logs	El sistema debe proporcionar registros y muestras de eventos que permitan probar y ajustar el algoritmo de detección de caídas.
RNF-15	Tolerancia a fallos	Los permisos ausentes, los contactos inexistentes, la ubicación no disponible o los fallos de red no deben bloquear la app; el sistema debe registrar el problema y continuar de forma segura cuando sea posible.
RNF-16	Compatibilidad	La app debe ejecutarse en dispositivos Android que admiten WIFI, notificaciones, SMS, llamadas, ubicación y vibración.
RNF-17	Escalabilidad	El sistema debe soportar varias redes WIFI guardadas y varios contactos de emergencia sin requerir cambios arquitectónicos.
RNF-18	Confianza del usuario	Las alertas, las cuentas atrás y las acciones de emergencia deben ser visibles y comprensibles para que el usuario sepa qué está haciendo el sistema.
RNF-19	Recuperación	Tras un corte de alimentación, pérdida de WIFI o reinicio, el dispositivo debe intentar recuperarse automáticamente usando la configuración guardada.

Tabla 2. Requisitos no funcionales del sistema.

Capítulo 4

Arquitectura y Diseño

4.1 Arquitectura del sistema

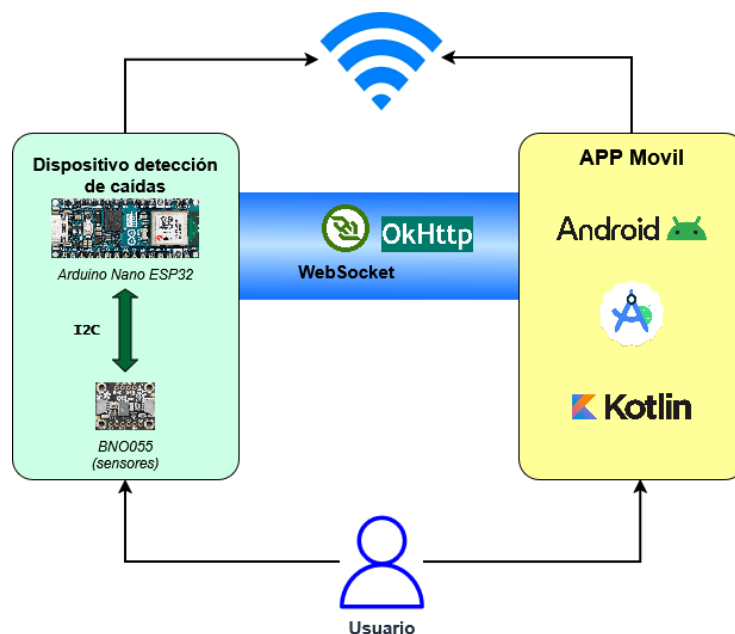


Ilustración 5. Arquitectura del Sistema

La arquitectura del sistema se ha diseñado como una solución IoT local formada por un dispositivo wearable de adquisición y análisis de datos, una aplicación móvil Android y una red WIFI doméstica que actúa como medio de comunicación entre ambos. A diferencia de una arquitectura cliente-servidor tradicional, el sistema no depende de un servidor externo para detectar caídas ni para coordinar el funcionamiento principal. El procesamiento de los datos de movimiento se realiza en el propio dispositivo wearable,

mientras que la aplicación móvil actúa como interfaz de usuario y como módulo de respuesta ante emergencias.

Desde el punto de vista lógico, el sistema se divide en dos módulos principales. El primero es el dispositivo de adquisición y análisis de datos, que puede considerarse el backend embebido del sistema. Este módulo está formado por el microcontrolador Arduino Nano ESP32, el sensor BNO055 y el firmware encargado de leer los datos del sensor, procesarlos, ejecutar el algoritmo de detección de caídas y comunicar los eventos detectados. El segundo módulo es la aplicación móvil, que actúa como dispositivo de interfaz hombre-máquina o HMI (Human-Machine Interface). Esta aplicación permite al usuario configurar el sistema, iniciar o detener la monitorización, visualizar información recibida del dispositivo, gestionar contactos de emergencia y ejecutar el flujo de alerta cuando se confirma una caída.

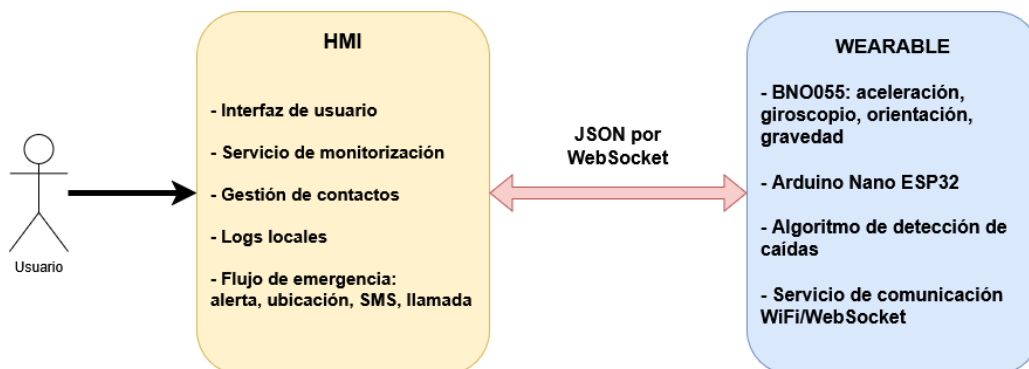


Ilustración 6. Arquitectura Lógica

En la Figura 6 se muestra la arquitectura lógica del sistema. El flujo principal comienza en el sensor BNO055, que obtiene información de movimiento del usuario. Estos datos son leídos por el Arduino mediante el bus I2C. La detección se realiza localmente en el microcontrolador, por lo que la aplicación móvil no necesita procesar continuamente los datos brutos para decidir si se ha producido una caída.

La aplicación móvil recibe los mensajes enviados por el dispositivo y los presenta al usuario. Además, mantiene un servicio de monitorización en primer plano para conservar la conexión con el Arduino mientras la monitorización está activa. Este

servicio se encarga de recibir los mensajes del dispositivo, registrar los datos localmente, actualizar la interfaz y activar el flujo de emergencia cuando llega una alerta de caída confirmada. En ese caso, la aplicación inicia una cuenta atrás durante la cual el usuario puede cancelar la alerta si se trata de una falsa alarma. Si la cuenta atrás finaliza sin cancelación, la aplicación obtiene la ubicación disponible del teléfono, envía un SMS a los contactos configurados y realiza una llamada al primer contacto de emergencia.

Desde el punto de vista físico, la arquitectura está formada por tres elementos principales: el wearable, el teléfono móvil y la red WIFI local. El wearable integra el firmware con el código de detección de caídas. El teléfono móvil ejecuta la aplicación Android y proporciona los recursos necesarios para la interacción con el usuario, la localización, las notificaciones, el envío de SMS y las llamadas. La red WIFI local permite la comunicación entre el dispositivo y la aplicación durante el funcionamiento normal del sistema.

En la Figura 7 se muestra la arquitectura de comunicación del sistema. La comunicación contempla dos modos de funcionamiento: modo de configuración y modo de monitorización normal. En el modo de configuración, el dispositivo crea un punto de acceso WIFI temporal cuando no dispone de credenciales guardadas o no puede conectarse a ninguna red conocida. El usuario conecta el teléfono móvil a esta red temporal y la aplicación envía las credenciales de la red doméstica al dispositivo mediante peticiones HTTP. En concreto, el dispositivo expone un servidor HTTP que permite consultar su estado y recibir las credenciales de red. Una vez recibida la configuración, el Arduino almacena las credenciales en memoria no volátil y se reinicia para intentar conectarse a la red WIFI indicada.

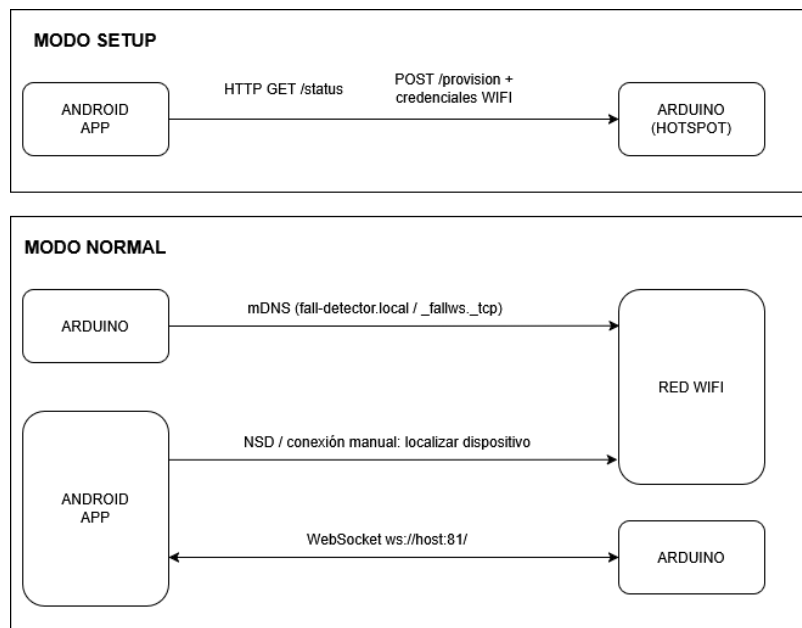


Ilustración 7. Arquitectura de Comunicación

En el modo de monitorización normal, el dispositivo se conecta como cliente a una de las redes WIFI guardadas. Después, anuncia su presencia en la red local mediante mDNS, utilizando un nombre de host y un servicio identificable por la aplicación móvil. La aplicación intenta descubrir automáticamente el dispositivo mediante el mecanismo NSD de Android. Si el descubrimiento automático no está disponible o falla, el usuario puede introducir manualmente la dirección IP o el nombre de host del dispositivo.

Una vez localizado el dispositivo, la aplicación abre una conexión WebSocket con el Arduino en el puerto configurado para la monitorización. Esta conexión WebSocket es una conexión persistente, es decir, permanece abierta mientras la monitorización está activa y permite que el dispositivo envíe mensajes a la aplicación en tiempo real sin que la aplicación tenga que realizar peticiones HTTP. Esto es adecuado para el sistema porque los eventos de caída deben comunicarse inmediatamente.

Los mensajes enviados por el dispositivo tienen formato JSON y se clasifican según su finalidad. Los mensajes de estado informan sobre la conexión y el arranque del dispositivo; los mensajes de depuración contienen métricas calculadas durante la

monitorización los mensajes de evento describen estados intermedios del algoritmo, como la detección de un impacto o una postura tumbada y los mensajes de caída indican que el algoritmo ha confirmado una caída. Cuando la aplicación recibe un mensaje de caída, inicia el flujo de alerta y respuesta de emergencia.

Esta arquitectura presenta varias ventajas para el proyecto. En primer lugar, reduce la dependencia de servicios externos, ya que la detección de caídas se realiza en el propio dispositivo y la comunicación con la aplicación ocurre dentro de la red local. En segundo lugar, separa claramente las responsabilidades de cada módulo: el wearable se encarga de adquirir y analizar los datos de movimiento, mientras que la aplicación móvil se encarga de la interacción con el usuario y de las acciones de emergencia.

4.2 Diseño del hardware

4.2.1 Decisiones técnicas

4.2.1.1 Elección de componentes

La selección de componentes del dispositivo se realizó a partir de las necesidades funcionales y físicas del sistema. Al tratarse de un dispositivo wearable para la detección de caídas, los componentes debían cumplir varios criterios: tamaño reducido, funcionamiento con batería, comunicación inalámbrica con la aplicación y capacidad suficiente para ejecutar el algoritmo de detección en tiempo real.

El sistema hardware se compone principalmente de cuatro bloques: el microcontrolador, el sensor inercial, el sistema de alimentación y la batería. El microcontrolador actúa como unidad central del dispositivo, ya que se encarga de leer los datos del sensor, procesarlos, ejecutar el algoritmo de detección de caídas y comunicarse con la aplicación móvil. El sensor inercial proporciona las mediciones para analizar el movimiento del usuario. El sistema de alimentación permite adaptar la tensión de la batería a la requerida por el circuito y gestionar la recarga. Finalmente, la batería permite que el dispositivo funcione de forma inalámbrica.

Para el microcontrolador se consideraron varias alternativas, entre ellas Arduino Uno Rev3, Arduino Uno Q, Raspberry Pi 4/5, Raspberry Pi Pico y Arduino Nano ESP32. Se descartaron el Arduino Uno Rev3, Arduino Uno Q, Raspberry Pi 4/5 y Raspberry Pi Pico, ya fuera por coste, capacidad de computación, tamaño, falta de características integradas o complejidad. Se seleccionó el **Arduino Nano ESP32** como microcontrolador principal. Esta placa integra conectividad WIFI y Bluetooth, mantiene un formato compacto y tiene una gran cantidad de librerías existentes. Además, su capacidad de procesamiento es suficiente para ejecutar el firmware necesario.

Para la medición del movimiento se seleccionó el sensor **Bosch BNO055**. El algoritmo de detección de caídas requiere información sobre aceleración, velocidad angular y orientación del dispositivo. El BNO055 es adecuado para este objetivo porque integra sensores para ello, además de proporcionar datos de orientación ya fusionados que ayudan a reducir la carga de procesamiento en el microcontrolador. También usa I2C para la comunicación, protocolo compatible con el Arduino Nano ESP32 y suficiente para la frecuencia de lectura requerida por el prototipo.

Un módulo de carga se encarga de controlar la recarga de la batería de forma segura. Por otro lado, un módulo boost transforma la tensión de la batería en una tensión de salida estable. En este proyecto el Arduino necesita 5 V, por lo que el sistema necesita elevar y estabilizar la tensión de la batería antes de suministrarla al microcontrolador. Para esta función se compararon dos alternativas: una combinación de módulos genéricos de carga y elevación de tensión, y el módulo Adafruit PowerBoost 500C. Los módulos genéricos tienen la ventaja de ser económicos, pero añaden más conexiones y pueden dar problemas de estabilidad en cargas reducidas/variables. Además, gran parte de los módulos apagan automáticamente la salida cuando hay un consumo bajo, y no es adecuado para este dispositivo que debe permanecer encendido. Por este motivo se seleccionó el **Adafruit PowerBoost 500C**. Este módulo integra en una misma placa la carga de una batería Li-Po y la conversión de tensión a una salida estable de 5,2 V. También incluye conexión para batería JST, entrada micro-USB de carga y pin de ENable para incorporar un interruptor de encendido y apagado. Así reduciendo el número de

componentes, simplificando el cableado y mejorando la robustez frente a una solución formada por varios módulos genéricos separados.

Respecto a la batería, se consideraron pilas de botón CR2032, baterías recargables AA o AAA y una batería Li-Po de 500 mAh a 3,7 V. Las pilas CR2032 se descartaron porque están orientadas a dispositivos de muy bajo consumo y no son adecuadas para alimentar de forma continua un microcontrolador con WiFi y un sensor inercial. Las baterías AA o AAA también podrían alimentar el wearable, pero requerirían varias celdas o un convertidor adicional, aumentando el tamaño del dispositivo. Finalmente, se seleccionó una batería **Li-Po de 500 mAh** de 3,7 V. Ofrece un equilibrio entre tamaño, peso, capacidad y compatibilidad con el módulo PowerBoost 500C. Aunque no proporciona la máxima autonomía posible, sí permite realizar pruebas funcionales del prototipo manteniendo un formato compacto.

4.2.2 Diseño de circuitos

Durante el desarrollo se realizaron dos propuestas de circuito. La primera versión utilizaba una combinación de módulos genéricos para la carga de la batería y la elevación de tensión. La segunda versión sustituyó esta solución por un módulo Adafruit PowerBoost 500C, que integra en una única placa la carga de la batería Li-Po y la conversión de tensión necesaria para alimentar el sistema. La evolución entre ambas versiones no se debió a un cambio de requisitos funcionales, sino a la necesidad de mejorar la estabilidad eléctrica, reducir la complejidad del cableado y facilitar la integración física dentro de la caja del dispositivo.

Versión inicial:

La figura 8 es la primera propuesta del circuito estaba formada por una batería Li-Po de 3,7 V, un módulo genérico de carga, un módulo elevador de tensión y una placa auxiliar con conector USB-C. La batería proporcionaba la energía del sistema, el módulo de carga permitía recargarla y el módulo elevador transformaba la tensión de la batería en una salida de 5 V para alimentar el Arduino Nano ESP32. El sensor BNO055 recibía alimentación desde el propio Arduino y se comunicaba con él mediante el bus I2C. Las

conexiones entre el Arduino y el sensor BNO055 para su comunicación, al igual que las conexiones para la alimentación del wearable se encuentran en el Anexo III.

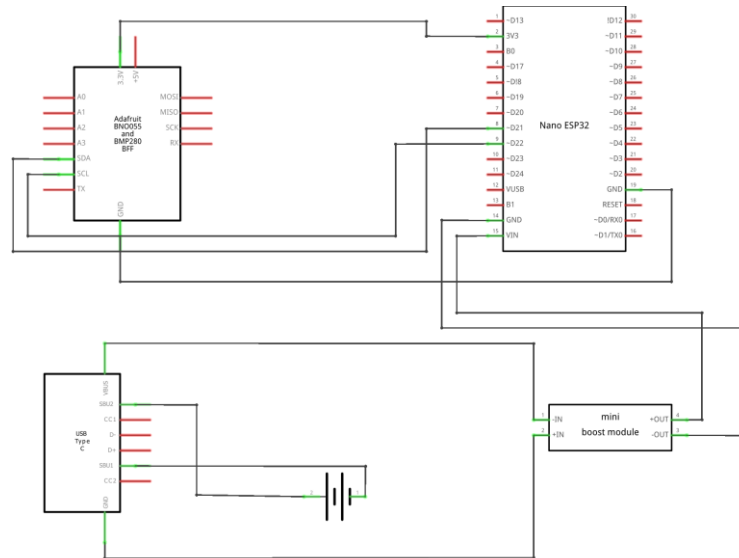


Ilustración 8. Esquemático de circuitos (primera versión)

Esta solución permitía obtener un prototipo funcional, pero presentó un problema importante durante las pruebas. El módulo elevador utilizado estaba diseñado para aplicaciones similares a baterías portátiles, en las que se espera un consumo mínimo en la salida. Cuando el consumo del circuito descendía por debajo de un umbral de aproximadamente 50 mA, el módulo desactivaba la salida de 5 V. Como consecuencia, el Arduino se apagaba y el sistema dejaba de monitorizar. Se identificó una posible solución técnica consistente en mantener activo el módulo mediante una señal aplicada al *K-pad*. Sin embargo, esta alternativa añadía una modificación manual al módulo, aumentaba la complejidad del montaje y reducía la robustez del dispositivo. Por ese motivo, se descartó como solución final. Aunque el primer circuito permitió validar la arquitectura básica del sistema, no resultaba adecuado para un dispositivo wearable que debe permanecer encendido de forma continua durante la monitorización y debe ser robusto ante impactos.

Versión final:

La figura 9 es la versión final del circuito utiliza el módulo PowerBoost 500C como bloque principal de alimentación. Este módulo se conecta a una batería Li-Po de 3,7 V mediante un conector JST y proporciona una salida regulada de aproximadamente 5.2 V para alimentar el Arduino. Además, el módulo incorpora el circuito de carga de la batería, que permite recargarla a través de su conector micro-USB.

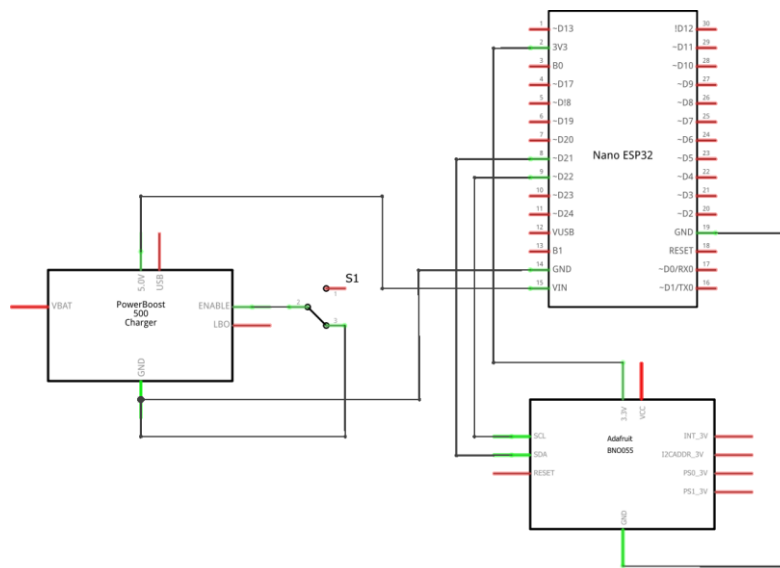


Ilustración 9. Esquemático de circuitos (versión final)

El Arduino Nano ESP32 se alimenta desde la salida de 5 V del PowerBoost 500C. La referencia de tierra del PowerBoost se conecta a la tierra del Arduino, de forma que todos los componentes del circuito comparten la misma referencia eléctrica. Esta conexión común es necesaria para garantizar el funcionamiento correcto tanto de la alimentación como de la comunicación entre el microcontrolador y el sensor. El PowerBoost 500C también permite conectar un interruptor físico al pin de *ENable*. En el diseño final, este interruptor se utiliza para activar o desactivar la salida de alimentación del dispositivo. De esta forma, el usuario puede encender o apagarlo sin desconectar la batería.

4.2.2.1 Custom PCB

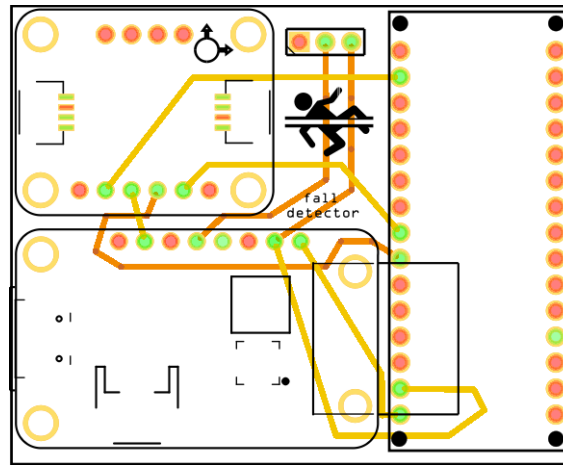


Ilustración 10. Circuito del PCB

Se ha diseñado una custom PCB para integrar las conexiones principales entre los componentes. La PCB permite fijar los componentes de forma más ordenada, reducir cableado interno y mejorar la resistencia del dispositivo. Además, facilita que el prototipo sea más compacto y cercano a una solución final de producto. Se pueden encontrar imágenes del PCB en el Anexo IV. Para proteger la PCB y sus componentes se diseñaron una cajas de impresión 3D. Los esquemas de los diseños la caja interna y externa, al igual que el diseño original previo al cambio a una custom PCB, se encuentran en el Anexo II.

4.3 Diseño del software

4.3.1 Decisiones técnicas

4.3.1.1 Elección de Lenguaje y Plataforma

Para el desarrollo de la aplicación móvil, la plataforma a elegir para su desarrollo solo tiene dos opciones reales, Android o IOS. En este proyecto se eligió el desarrollo de una aplicación HMI para las plataformas Android, ya que es el sistema operativo para móviles más usado [63].

Frente a una solución multiplataforma o a una aplicación web, el desarrollo nativo en Android permite una integración más directa con las APIs del sistema. La aplicación no solo muestra información al usuario, también debe mantener una conexión activa con el dispositivo de detección, recibir eventos en tiempo real, registrar logs localmente y ejecutar acciones de emergencia. Por ello, una aplicación Android resulta más adecuada que una aplicación web.

El lenguaje elegido para la aplicación móvil fue **Kotlin**. Kotlin es un lenguaje moderno, compatible con el ecosistema Android y adecuado para desarrollar aplicaciones con una estructura mantenible. Su uso permite implementar de forma ordenada las distintas responsabilidades de la aplicación, como la interfaz de usuario, el servicio de monitorización, la gestión de contactos, el almacenamiento local y el flujo de alerta ante caídas. Además, su interoperabilidad con las bibliotecas del ecosistema Android facilita la integración con APIs del sistema y con librerías de comunicación.

El firmware se encarga de inicializar el hardware, leer periódicamente el sensor BNO055, calcular las métricas necesarias para el algoritmo de detección de caídas, gestionar la conexión WIFI y enviar mensajes a la aplicación móvil. El lenguaje utilizado es **C++**, ya que es el lenguaje habitual de desarrollo para microcontroladores Arduino y permite controlar directamente los recursos del dispositivo.

4.3.1.2 Elección de Protocolo de Comunicación

La comunicación entre el dispositivo y la aplicación móvil se diseñó como una arquitectura cliente-servidor local sobre una red WIFI doméstica. En este modelo, el Arduino Nano ESP32 y el teléfono móvil se conectan a la misma red, actuando el ESP32 como servidor local y la aplicación Android como cliente. Esta solución resulta adecuada para un entorno doméstico, ya que permite que ambos dispositivos se comuniquen dentro de la cobertura de la red WIFI sin depender de un servidor externo.

El sistema tiene dos modos de comunicación. En el modo de configuración inicial, el Arduino crea un hotspot WIFI temporal cuando no tiene credenciales guardadas. La aplicación se conecta y envía las credenciales de la red doméstica mediante peticiones

HTTP. Una vez recibidas, el dispositivo las almacena en memoria no volátil y se reinicia para conectarse a la red.

En el modo normal de monitorización, la comunicación se realiza mediante WebSocket para mantener una conexión persistente entre la aplicación y el Arduino, de forma que el dispositivo puede enviar mensajes en tiempo real sin peticiones periódicas. Esto es especialmente importante para comunicar eventos de caída, estados del algoritmo y mensajes de debug con el menor retardo posible.

Para facilitar la conexión, el dispositivo anuncia su presencia en la red local mediante descubrimiento de servicio. La aplicación intenta localizarlo automáticamente y si falla, permite introducir manualmente la dirección IP o el nombre de host. Esto mejora la usabilidad y proporciona un mecanismo de respaldo ante posibles fallos de descubrimiento.

Por otro lado, la alternativa es Bluetooth. Pero teniendo en cuenta que su señal depende de la distancia entre el dispositivo y el móvil, en el hogar es más común dejar el móvil en una zona en la que no esté el usuario, lo que da riesgo a que no llegue bien la conexión (también porque puede verse muy afectada por otras interferencias). Por ello se vio más adecuado el uso de WIFI.

Pero pensando en el uso del dispositivo para fuera del hogar, el concepto es diferente. Se planteó la idea de que se usara WIFI para el hogar y Bluetooth para el exterior. Pero el uso de estos dos protocolos complicaba mucho la arquitectura del proyecto, y hacía empeorar el consumo de la batería. Por ello se usa la red WIFI que genera el teléfono móvil con conexión de datos usándolo como un hotspot. De esta forma la arquitectura no cambia, solo hace falta activar los datos móviles y tener metidas las credenciales para su uso. Y así manteniendo la duración de la batería durante un periodo más largo.

4.3.2 Diseño del algoritmo

El algoritmo de detección de caídas se diseñó como un algoritmo determinista basado en umbrales y estados. Su objetivo es identificar una caída a partir de la combinación de

varias señales de movimiento, evitando depender únicamente de un pico de aceleración. Esta decisión se debe a que muchas actividades normales, como sentarse rápidamente, saltar o dejar caer el dispositivo sobre una superficie, pueden producir aceleraciones elevadas sin corresponder necesariamente a una caída real.

El diseño parte de una interpretación por fases del evento de caída. De forma simplificada, una caída puede incluir una pérdida de equilibrio o soporte, una rotación del cuerpo, un impacto, un cambio de postura y un periodo posterior de poca actividad. Por ello, el algoritmo no confirma una caída en el primer instante de movimiento brusco, sino que exige una secuencia de condiciones: primero debe aparecer un evento candidato asociado a alta energía de movimiento y rotación, después, el dispositivo debe quedar en una postura compatible con una persona tumbada, y finalmente, debe mantenerse un periodo de inactividad.

Para ello se utilizan cuatro tipos de datos proporcionados por el sensor BNO055: aceleración lineal, velocidad angular del giroscopio, orientación y vector de gravedad. La aceleración lineal y la velocidad angular se emplean para detectar movimientos bruscos y rotaciones. El vector de gravedad se utiliza para estimar la postura del usuario, ya que permite calcular si el eje vertical del cuerpo se encuentra aproximadamente alineado con la gravedad o perpendicular a ella. La orientación Euler se registra como información adicional para análisis posterior, pero la decisión principal de postura se basa en el vector de gravedad.

A partir de la aceleración lineal se calcula su magnitud escalar:

$$AccMag = \sqrt{ax^2 + ay^2 + az^2}$$

De forma equivalente, a partir de la velocidad angular del giroscopio se calcula la magnitud de rotación:

$$GyroMag = \sqrt{gx^2 + gy^2 + gz^2}$$

Estas dos magnitudes permiten trabajar con valores independientes de la orientación exacta del sensor en cada instante. La magnitud de aceleración representa la intensidad del movimiento lineal, mientras que la magnitud del giroscopio representa la intensidad de la rotación del dispositivo.

Para combinar ambas variables se define una métrica denominada **fallEnergy**. Esta métrica no representa una energía física expresada en julios, es un indicador diseñado para agrupar aceleración y rotación en un único valor de decisión:

$$Fall\ Energy = AccMag^2 + 0,01 \cdot GyroMag^2$$

La aceleración y la velocidad angular se elevan al cuadrado para que los movimientos más intensos tengan mayor peso en la métrica. El factor 0,01 se aplica a la componente del giroscopio porque sus valores se expresan en grados por segundo y pueden ser numéricamente mucho mayores que los valores de aceleración lineal. Sin esta ponderación, la rotación dominaría la métrica y reduciría la utilidad de la aceleración en la detección.

En lugar de utilizar únicamente el valor instantáneo de *fallEnergy*, el algoritmo mantiene una ventana móvil de 20 muestras. Dado que el periodo de muestreo es de aproximadamente 40 ms, la frecuencia de muestreo es cercana a 25 Hz, por lo que esta ventana representa aproximadamente 0,8 segundos de movimiento. La energía total se calcula como la suma de las últimas 20 muestras:

$$Total\ Energy = \sum Fall\ Energy_i$$

Esta ventana móvil permite detectar caídas que no se manifiestan en una única muestra, sino como una combinación breve de aceleración, rotación e impacto. De este modo, el algoritmo es menos dependiente de un único pico aislado.

Para la estimación de la postura se utiliza el vector de gravedad. El dispositivo se coloca de forma que el eje Y del sensor corresponda aproximadamente al eje vertical del cuerpo del usuario. A partir de la componente de gravedad sobre ese eje y de la magnitud del vector de gravedad se calcula el ángulo de postura:

$$\text{Angulo Postura} = \arccos\left(\frac{|gY|}{|g|}\right)$$

Un ángulo cercano a 0° indica que el eje vertical del cuerpo está alineado con la gravedad, lo que corresponde a una postura de pie o sentada. Un ángulo cercano a 90° indica que dicho eje es perpendicular a la gravedad, lo que corresponde a una postura tumbada. Antes de utilizar esta medida, se comprueba que la magnitud del vector de gravedad esté en un rango válido entre 7,0 m/s² y 12,5 m/s². Si la lectura queda fuera de ese intervalo, se considera poco fiable y se conserva el estado de postura anterior.

Para reducir cambios bruscos provocados por ruido o lecturas puntuales, el ángulo de postura se filtra mediante un suavizado exponencial:

$$\text{Postura Filtrada} = 0,8 \cdot \text{Postura Filtrada Anterior} + 0,2 \cdot \text{Postura Actual}$$

Sobre este ángulo filtrado se define el estado de postura tumbada mediante histéresis. El sistema entra en estado tumbado cuando el ángulo filtrado es igual o superior a 65°, y sale de ese estado cuando el ángulo baja hasta 45° o menos. El uso de dos umbrales evita oscilaciones cuando el ángulo se encuentra cerca del límite de decisión.

El estado de inactividad se calcula utilizando aceleración y velocidad angular. El usuario se considera inactivo cuando se cumplen simultáneamente las siguientes condiciones:

$$\text{AccMag} < 0,5 \text{ AND GyroMag} < 20,0$$

Esta condición debe mantenerse durante 3 segundos para que el algoritmo considere que existe inactividad posterior al evento candidato. Esta restricción evita confirmar una caída si el usuario continúa moviéndose, por ejemplo porque se está levantando, girando o recolocando después de un movimiento brusco.

El comportamiento global del algoritmo se modela como una máquina de estados. En la Figura 11 se muestra el flujo de decisión seguido durante la monitorización.

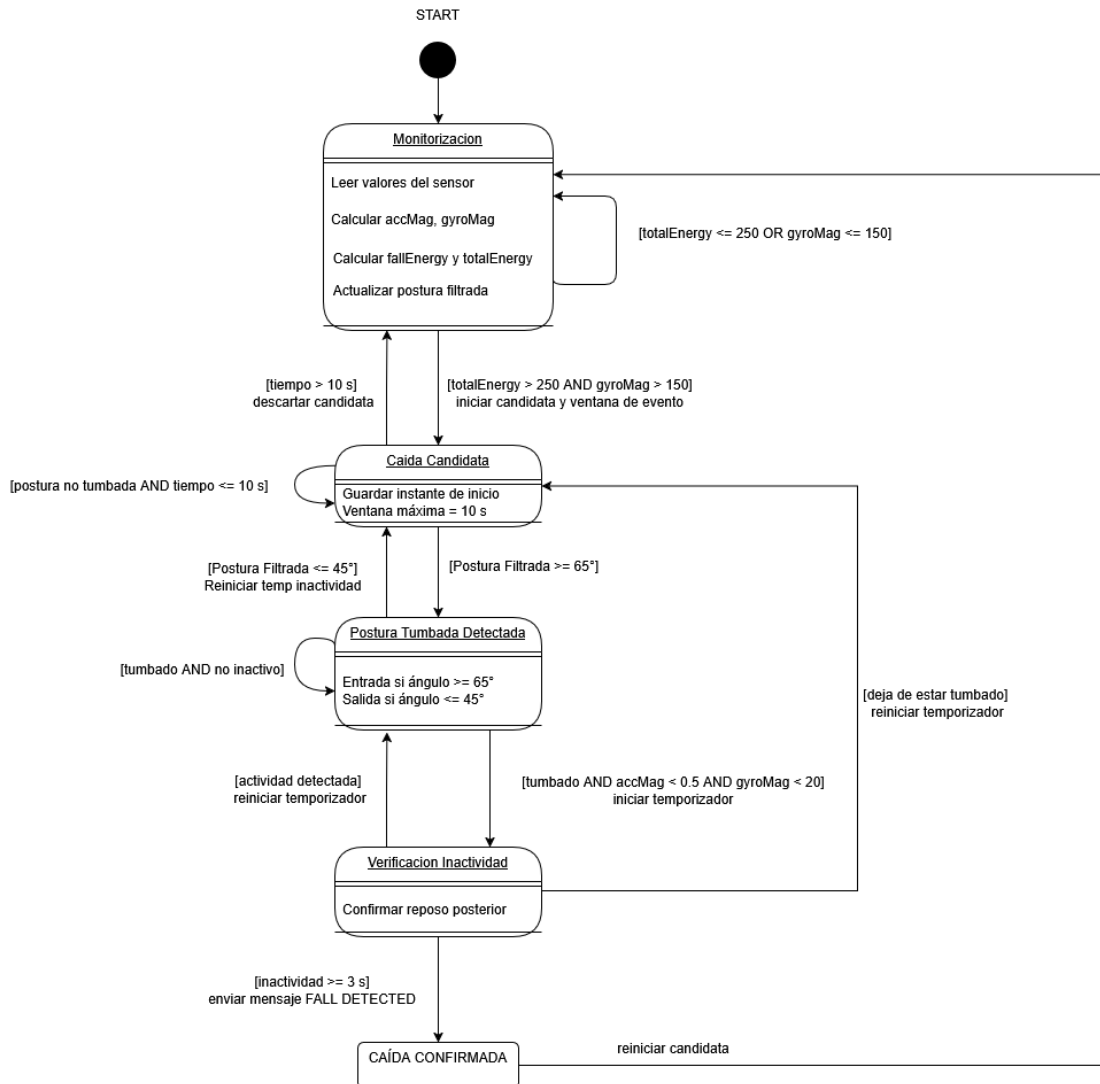


Ilustración 11. Diagrama State Machine del algoritmo

Desde el estado de monitorización normal, el sistema lee periódicamente el sensor y calcula las magnitudes anteriores. Cuando la total energy supera el umbral de 250 y la magnitud del giroscopio supera el umbral de 150, el algoritmo no confirma todavía una caída, sino que crea una caída candidata. Esta condición indica que se ha producido un movimiento suficientemente intenso y con rotación significativa.

Una vez creada la caída candidata, se abre una ventana temporal máxima de 10 segundos. Durante ese tiempo, el algoritmo comprueba si el usuario queda en postura tumbada y posteriormente inactivo. Si ambas condiciones se cumplen y la inactividad se mantiene durante al menos 3 segundos, la caída se confirma. En ese momento, el Arduino envía a la aplicación móvil un mensaje de alerta de tipo fall.

Si transcurren los 10 segundos sin que se cumplan las condiciones de postura e inactividad, la caída candidata se descarta y el sistema vuelve al estado de monitorización normal. También se reinicia el proceso si el usuario muestra actividad suficiente como para no cumplir la condición de inactividad. De esta forma, el algoritmo reduce falsos positivos provocados por movimientos bruscos que no terminan en una postura tumbada e inmóvil.

Además de la lógica de detección, el algoritmo registra una event window cuando aparece una caída candidata. Esta ventana almacena muestras anteriores y posteriores al instante de activación, con el objetivo de facilitar el análisis posterior de los datos y el ajuste de umbrales. La ventana de evento no participa directamente en la confirmación de la caída, su función es proporcionar información para debug, validación y posibles mejoras futuras del algoritmo.

Los valores usados para los umbrales se tomaron como punto de partida a partir de estudios académicos sobre detección de caídas basada en sensores inerciales, especialmente aquellos que combinan aceleración, velocidad angular, energía de movimiento y postura. Posteriormente, estos valores fueron ajustados mediante pruebas propias para adaptarlos al algoritmo implementado, al sensor utilizado y al posicionamiento físico del dispositivo [10][11][13][16][21][24][31].

Capítulo 5

Implementación, despliegue y validación

5.1 Detalles de implementación

5.1.1 Clases de la Aplicación Movil

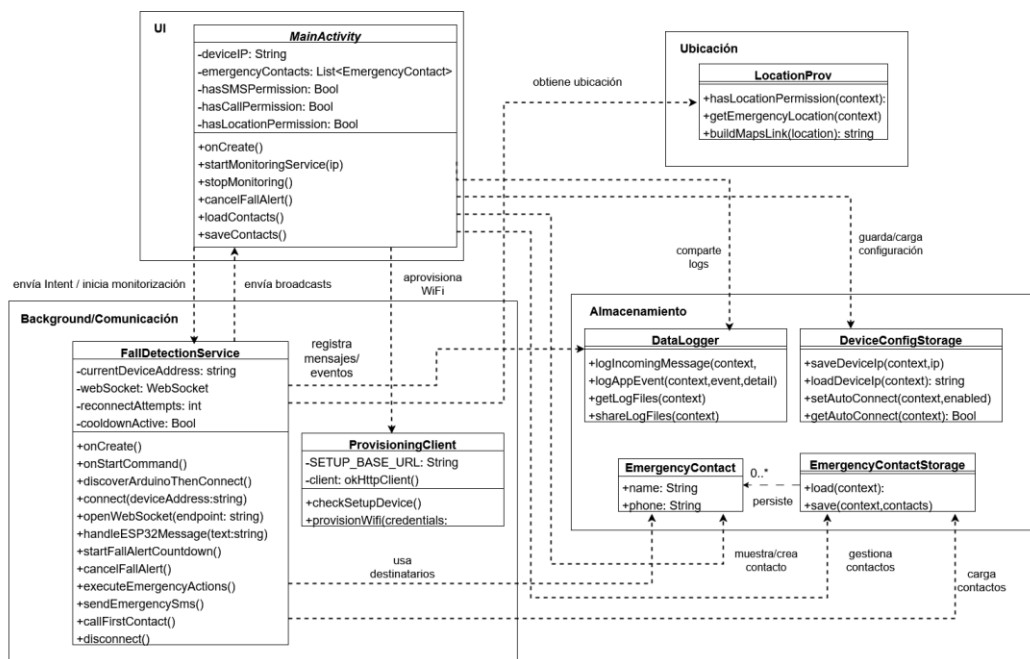


Ilustración 12. Class Diagram

En la figura 12 se representa la estructura de la aplicación móvil. Desde el punto de vista de implementación, las clases se organizan en cuatro componentes funcionales: presentación y control de la aplicación, comunicación y monitorización en segundo plano, persistencia local y obtención de ubicación.

MainActivity es la clase principal de la interfaz del usuario. Es el punto de entrada de la aplicación y se encarga de acciones como empezar o parar la monitorización, cancelar

una alerta de caída, administrar los contactos de emergencia y acceder al proceso de configuración del dispositivo (provisionamiento). También mantiene el estado de los permisos necesarios para el funcionamiento de la aplicación (SMS, llamadas, notificaciones y ubicación). Desde esta clase se inicia o detiene *FallDetectionService*, se cargan y guardan contactos con *EmergencyContactStorage*, se almacena la dirección del dispositivo de detección mediante *DeviceConfigStorage* y se permite compartir los logs generados por *DataLogger*.

FallDetectionService es la clase que se encarga de la ejecución background. Esta clase mantiene la conexión con el Arduino, gestiona el websocket, recibe los mensajes enviados por el Arduino y procesa los mensajes de caída. Cuando recibe un mensaje de tipo *fall*, inicia la cuenta atrás de alerta, activa la vibración y el sonido, y permite cancelar la alerta si se trata de una falsa alarma. Si la cuenta atrás finaliza, el servicio ejecuta las acciones de emergencia: obtiene la ubicación, envía SMS a los contactos de emergencia y llama al primer contacto registrado.

ProvisioningClient se encarga de la comunicación durante la fase de provisionamiento del dispositivo (con HTTP). Su función es comprobar si el Arduino está disponible en modo hotspot (haciendo una petición a */status*) para enviarle las credenciales WIFI introducidas por el usuario. Se utiliza la dirección base *http://192.168.4.1* que corresponde al punto de hotspot creado por el dispositivo. No participa en la detección de caídas en tiempo real, solo se encarga de la configuración inicial de la conectividad WIFI del Arduino.

DeviceConfigStorage se encarga de guardar la configuración del dispositivo en la aplicación. Su responsabilidad es guardar y cargar la dirección IP o hostname del Arduino (y la opción de conexión automática). Así la aplicación puede recordar la configuración utilizada por el usuario y no necesita solicitar la dirección del dispositivo cada vez que se ejecuta. Esta clase es una capa sencilla de almacenamiento local para parámetros de configuración.

EmergencyContact es el modelo de un contacto de emergencia. Contiene los datos necesarios para ejecutar las acciones de emergencia: el nombre del contacto y su número de teléfono. Esta clase se utiliza por la interfaz de usuario, para mostrar y administrar la lista de contactos, y por *FallDetectionService*, para enviar SMSs y realizar la llamada de emergencia.

EmergencyContactStorage es la clase que guarda y carga la lista de contactos de emergencia. Es una capa de persistencia para *EmergencyContact*, así permite que los contactos introducidos por el usuario se guarden entre ejecuciones de la aplicación. *MainActivity* utiliza esta clase para mostrar y modificar los contactos desde la interfaz, y *FallDetectionService* la utiliza para recuperar los contactos cuando debe ejecutar las acciones de emergencia.

DataLogger se encarga de registrar en local la información generada durante el funcionamiento de la aplicación. Se guardan los mensajes recibidos desde el Arduino, los eventos de la aplicación y los datos útiles de debug y validación del sistema. Su uso es especialmente importante durante la fase de pruebas, ya que permite analizar posteriormente los mensajes para hacer el estudio estadístico. También da funcionalidad para obtener y compartir los archivos de log.

LocationProv se encarga de obtener la ubicación del teléfono y formatearla. Comprueba si la aplicación tiene los permisos de ubicación, consigue una ubicación disponible durante una emergencia y genera un enlace de Google Maps usando las coordenadas. Esta clase es utilizada por *FallDetectionService* cuando una caída no es cancelada por el usuario y se tienen que enviar SMSs con la ubicación aproximada a los contactos de emergencia.

Las librerías usadas para la aplicación se encuentran en el Anexo I.

5.1.2 Aprovisionamiento de red

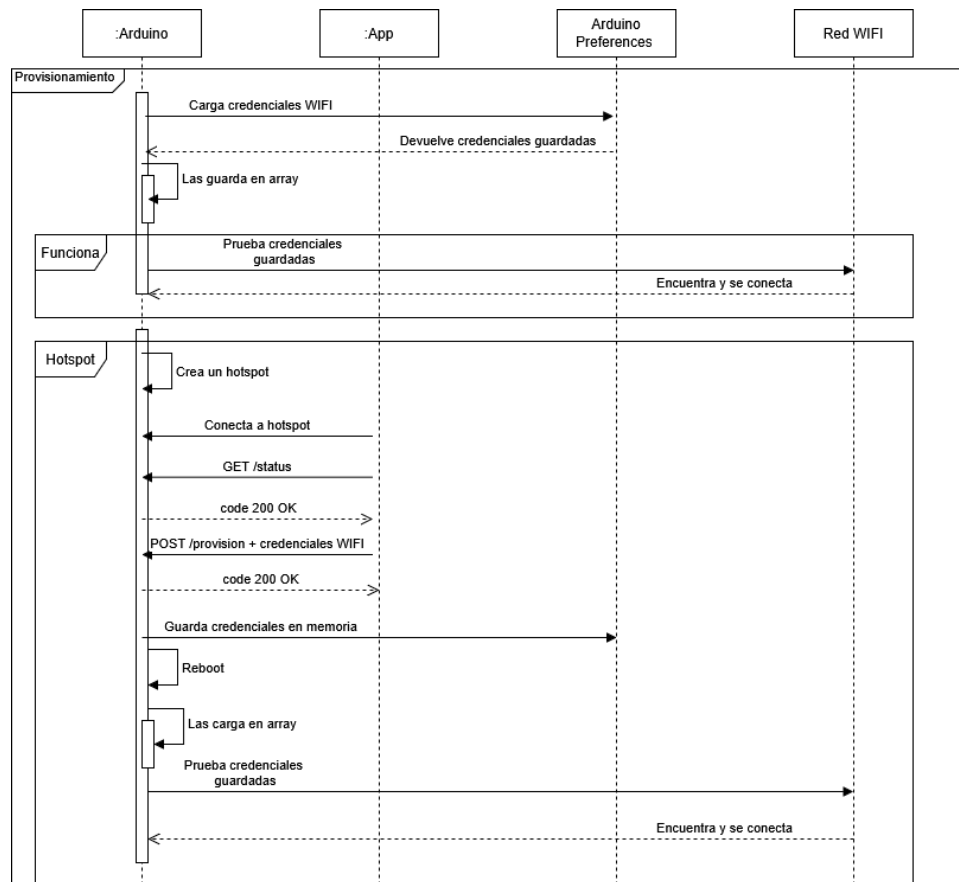


Ilustración 13. Sequence Diagram - Aprovisionamiento WIFI

En la figura 13 se representa la fase de aprovisionamiento WIFI del dispositivo, con el objetivo de permitir que el dispositivo pueda conectarse automáticamente a una red conocida o en caso contrario, ofrecer un mecanismo de setup inicial desde la aplicación móvil.

Primero, al arrancar el dispositivo, el Arduino accede a la memoria de preferencias para cargar las credenciales WIFI almacenadas previamente. Estas credenciales se guardan temporalmente en un array interno para poder ser probadas de una en una, teniendo preferencia el orden dentro del array (prueba de una en una, en el orden en el que estén). A continuación, el dispositivo intenta conectarse a las redes guardadas. Si alguna

de ellas está disponible y las credenciales son correctas, el Arduino establece la conexión con la red WIFI y puede continuar con el modo normal de funcionamiento.

Si ninguna red guardada funciona, se entra en modo de aprovisionamiento. En este estado, el Arduino crea un punto de acceso propio temporal (WIFI hotspot). El usuario conecta manualmente el teléfono móvil a esta red WIFI de configuración, así permitiendo que la aplicación se comuniqué directamente con el dispositivo. Una vez establecida la conexión, la aplicación realiza una petición *GET /status* para comprobar que el Arduino está disponible y preparado para recibir la configuración. Si el Arduino responde con un código *200 OK*, la aplicación envía una petición *POST /provision* que contiene las nuevas credenciales WIFI introducidas por el usuario dentro de la aplicación.

Cuando el Arduino recibe la petición de aprovisionamiento, este almacena las credenciales en la memoria persistente mediante el sistema de preferencias. De esta forma, los datos de conexión permanecen guardados aunque el dispositivo se reinicie o pierda alimentación. Una vez finalizado el guardado, el Arduino responde con un nuevo código *200 OK* y se reinicia.

Después de reiniciarse, se empieza de nuevo. El Arduino carga las credenciales desde memoria, las guarda en el array interno y vuelve a intentar la conexión con las redes disponibles. En este caso, al haberse almacenado correctamente la nueva configuración, el Arduino encuentra la red y se conecta a ella. De esta manera se permite que se tenga un comportamiento autónomo en el uso normal, pero también que haya una manera de configuración sencilla cuando no existe una red válida guardada.

5.1.3 Flujo de Monitorización

En la figura 14 se representa la fase de monitorización del sistema, correspondiente al “modo normal” de funcionamiento una vez que el Arduino ya tiene una conexión WIFI válida. En esta fase, el objetivo es establecer un canal de comunicación entre el dispositivo de detección de caídas y la aplicación móvil, permitiendo el envío de mensajes en tiempo real.

Monitorización (Modo Normal)

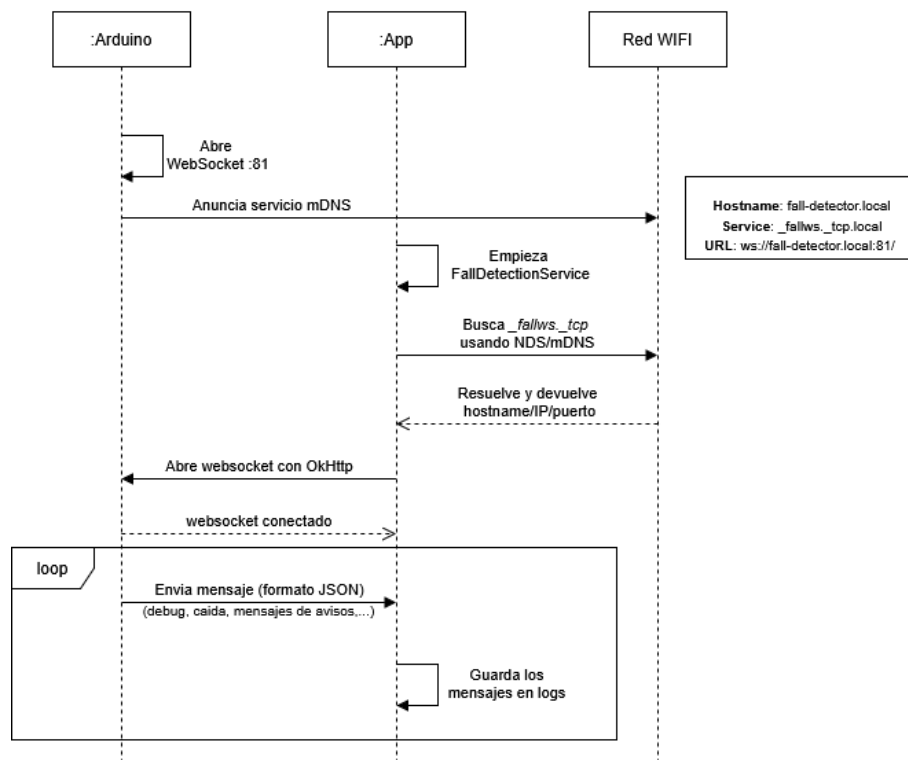


Ilustración 14. Sequence Diagram - Monitorización (Conexión Arduino/Móvil)

Primero el Arduino inicia el servidor WebSocket en el puerto 81. Este servidor es el canal utilizado para enviar mensajes a la aplicación, como información de debug, eventos intermedios, avisos de estado y alertas de caída. Después el dispositivo anuncia su disponibilidad en la red local mediante mDNS, publicando el nombre `fall-detector.local` y el servicio `_fallws._tcp.local`. De esta manera el usuario no necesita conocer cual es la dirección IP del Arduino, lo localiza dinámicamente dentro de la red WIFI (Aún está configurado en la aplicación móvil la posibilidad de introducir la IP del dispositivo a mano, pero la opción predeterminada es usar el nombre del servicio).

En la aplicación móvil, se inicia el servicio *FallDetectionService*, que se encarga de ejecutar la monitorización en segundo plano. Hace una búsqueda en la red local utilizando Android NSD/mDNS para encontrar el servicio de `_fallws._tcp`. Si encuentra el servicio anunciado por el Arduino, la red devuelve a la aplicación la información

necesaria para establecer la conexión (hostname, la dirección IP y el puerto correspondiente).

Cuando ya está resuelta la dirección IP del dispositivo, la aplicación abre una conexión WebSocket. Cuando la conexión se establece correctamente, queda habilitado un canal persistente entre el Arduino y la aplicación. Y ya a partir de ese momento, el Arduino envía de forma continua mensajes en formato JSON a través del WebSocket haciendo un broadcast.

Finalmente, cada mensaje recibido por la aplicación se almacena en el sistema de logs. Este registro permite conservar un historial de la comunicación con el Arduino.

5.1.4 Flujo de detección de caídas

En la Figura 15 se muestra el diagrama de secuencia correspondiente al flujo de detección de caídas. En este proceso intervienen el sensor BNO055, el Arduino, la aplicación móvil, el usuario y los contactos de emergencia. Esta fase se ejecuta durante el modo normal de monitorización, cuando el dispositivo y la aplicación ya se encuentran conectados.

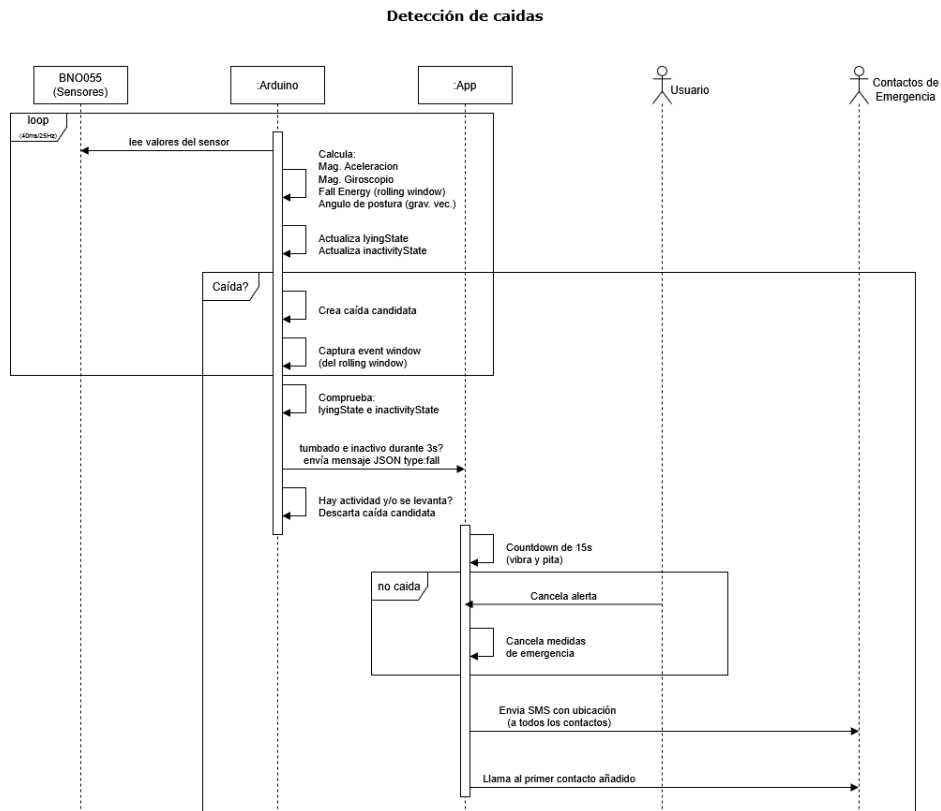


Ilustración 15. Sequence Diagram - Detección de Caídas

El flujo comienza en el dispositivo de detección de caídas. El Arduino lee periódicamente los datos del sensor BNO055 y ejecuta el algoritmo de detección (apartado 4.3.3). El Arduino puede generar distintos mensajes hacia la aplicación: mensajes de debug, eventos, ventanas de muestras o una alerta de caída confirmada.

Cuando se confirma una caída, el Arduino envía a la aplicación móvil un mensaje JSON de tipo *fall*. A partir de este momento, la lógica pasa a ser gestionada por *FallDetectionService*, el servicio de monitorización de la aplicación.

Al recibir la alerta de caída, en *FallDetectionService* inicia una cuenta atrás de 15 segundos. Durante este intervalo, la aplicación avisa al usuario mediante vibración, pitidos y un mensaje en pantalla. El objetivo de esta cuenta atrás es permitir que el usuario cancele el proceso si es una falsa alarma. Si el usuario pulsa la opción de

cancelación antes de que finalice el temporizador, el servicio detiene la alerta y no ejecuta las acciones de emergencia.

Si la cuenta atrás finaliza sin cancelación del usuario, la aplicación ejecuta el flujo de emergencia. Primero intenta obtener la ubicación actual del teléfono o, si no está disponible, la última ubicación conocida. A partir de estas coordenadas genera un enlace de Google Maps, que se incluye en el SMS de emergencia.

A continuación, la aplicación carga los contactos de emergencia configurados por el usuario. El sistema envía un SMS a cada contacto guardado, incluyendo el aviso de caída y el enlace de ubicación. Después, realiza una llamada telefónica al primer contacto de la lista en modo altavoz para facilitar la comunicación en caso de que el usuario no esté cerca o no pueda sujetar el teléfono después de la caída.

Durante todo el proceso, la aplicación registra los eventos relevantes en los logs locales.

5.1.5 Funcionamiento del algoritmo

Durante el modo normal de funcionamiento, el método *loop()* comprueba que el dispositivo continúe conectado a la red WIFI. Si la conexión se mantiene activa, se ejecuta periódicamente *runFallDetection()*. En caso de pérdida de conexión, el dispositivo detiene el servicio mDNS y se reinicia para intentar recuperar la conexión utilizando las redes WIFI guardadas.

La función *runFallDetection()* realiza una iteración completa del algoritmo. En primer lugar, ejecuta *webSocket.loop()* para mantener activa la comunicación con la aplicación móvil. Después obtiene del BNO055 los datos necesarios. A partir de estas lecturas se calculan las magnitudes necesarias para la detección y se actualizan las variables internas del algoritmo.

El muestreo se realiza cada 40 ms, lo que equivale aproximadamente a 25 muestras por segundo. Para calcular la Total Energy se utiliza una rolling window de 20 posiciones,

declarado como *energyBuffer*. Esta se comporta como una FIFO y se recalcula la suma de la ventana. Esto permite mantener un análisis sin almacenar todos los datos.

La lógica de caída candidata se implementa mediante varias variables de estado. La variable *possibleFall* indica si existe una caída candidata activa. La variable *fallTime* almacena el instante en el que se detectó el evento inicial. Las variables *lyingStart* e *inactiveStart* guardan el inicio de la postura tumbada y del periodo de inactividad respectivamente. La función *resetFallCandidate()* reinicia estas variables cuando la caída se confirma, expira o descarta.

La postura tumbada se gestiona mediante la función *updateLyingPostureState()*. El filtrado del ángulo se realiza en *updatePostureFilter()*, evitando que pequeñas variaciones de lectura provoquen cambios bruscos de estado. Antes de calcular el ángulo de postura, la función *computeMountedPostureAngleDeg()* comprueba que la magnitud del vector de gravedad se encuentre dentro de un rango válido. Si la lectura no es fiable, se conserva el estado anterior de postura.

Cuando la total energy y la magnitud del giroscopio superan los umbrales definidos, el firmware activa una caída candidata. En ese momento se envía a la aplicación un mensaje de evento y se inicia la captura de una event window mediante *startEventCapture()*. No forma parte de la decisión de caída, se utiliza para registrar datos antes y después del evento y facilitar el análisis posterior.

La ventana de evento se implementa mediante dos buffers. El primero, *ringBuffer*, conserva las muestras anteriores al instante de activación. El segundo, *eventBuffer*, almacena la ventana completa del evento, formada por muestras previas y posteriores al trigger. Cuando la captura termina, *finishEventCapture()* envía a la aplicación un mensaje JSON de tipo *event_window* con las muestras registradas.

Si existe una caída candidata activa, se comprueba si esta supera el tiempo máximo permitido. Si transcurren 10 segundos sin cumplir las condiciones, la candidata se descarta. En caso contrario, se evalúa la postura e inactividad durante el tiempo

requerido. Cuando la inactividad se mantiene durante 3 segundos, se confirma la caída y se envía a la aplicación el mensaje: `{"type":"fall","message":"FALL DETECTED"}`.

Después de enviar la alerta, el firmware reinicia las variables de la caída candidata y vuelve al estado normal de monitorización. Además de los mensajes de caída, el dispositivo envía mensajes de debug con valores como aceleración, giroscopio, energy, ángulo de postura, estado de postura tumbada, estado de inactividad y calibración del sensor. Estos mensajes permiten comprobar el comportamiento del algoritmo.

Por último, el firmware gestiona la calibración del BNO055 mediante memoria no volátil. Así se reduce la necesidad de recalibrar manualmente el sensor cada vez que se enciende el dispositivo.

5.2 Plan de pruebas

5.2.1 Pruebas de precisión del algoritmo

Se han llevado a cabo un total de 100 pruebas, 30 caídas simuladas y 70 pruebas sin caída. Las pruebas sin caída incluyen actividades normales de la vida diaria y movimientos similares a una caída diseñados para poner a prueba el algoritmo. Las pruebas de caída incluyeron caídas hacia delante, hacia atrás, laterales hacia la izquierda, laterales hacia la derecha, colapsos y caídas desde una posición sentada. 5 pruebas de cada.

Los resultados obtenidos de la validación del sistema se presentan en la tabla 3:

Métrica	Resultado	Significado
Accuracy	99.0%	% general de clasificación correcta
Recall	100.0%	% de caídas reales detectadas
Specificity	98.6%	% de no-caídas correctamente ignoradas
Precision	96.8%	% de alertas de caídas que eran reales
Negative predictive value	100.0%	% de clasificados como negativos que son negativos
F1 score	98.4%	Balance entre Precisión y Recall
False positive rate	1.4%	% de pruebas no-caída detectados como caída
False negative rate	0.0%	% de caídas no detectadas

Tabla 3. Resultados de la validación.

Se identificaron correctamente las 30 caídas simuladas, obteniendo una sensibilidad del 100 %. De las 70 pruebas sin caída, 69 fueron clasificadas correctamente como no caídas y una produjo un falso positivo. Como resultado, el sistema obtuvo una especificidad del 98,6 %, una precisión del 96,8 %, una accuracy global del 99,0 % y una puntuación F1 del 98,4 %. No se observaron falsos negativos durante el estudio, ninguna de las caídas fue ignorada por el sistema.

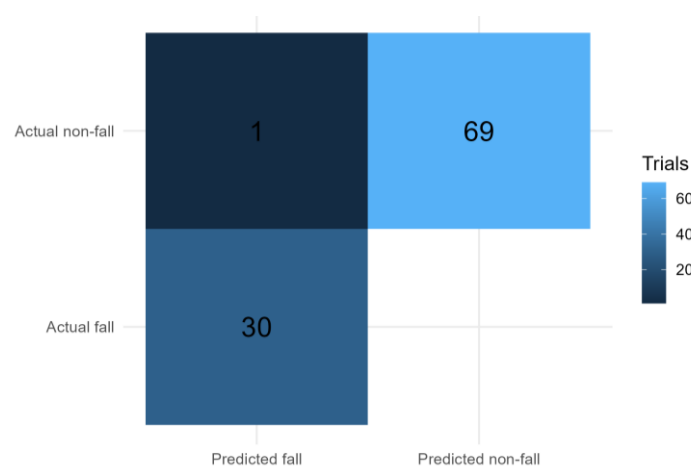


Ilustración 16. Matriz de confusión.

La figura 16 muestra 30 verdaderos positivos, 69 verdaderos negativos, 1 falso positivo y 0 falsos negativos. El único error de clasificación ocurrió durante una prueba de tirarse al sofá. Esta prueba estaba etiquetada como no caída, pero fue detectada como caída. Este tipo de movimiento es especialmente difícil de distinguir, ya que puede generar una aceleración fuerte, un cambio repentino de postura y un periodo de inactividad posterior, lo que lo hace muy similar a una caída real.

La figura 17 es el análisis por tipo de actividad y muestra que todas las categorías de caída fueron detectadas en el 100 % de los casos. Las actividades normales de la vida diaria, como estar de pie, caminar, caminar rápido, sentarse, levantarse, tumbarse normalmente, levantarse de la cama e inclinarse para recoger un objeto, no produjeron ninguna alerta de caída. Entre las actividades similares a una caída, los golpes al dispositivo, los saltos, sentarse rápidamente y los tropiezos con recuperación también fueron correctamente ignorados. El único falso positivo se produjo en la actividad de dejarse caer sobre el sofá, donde 1 de 5 pruebas generó una alerta de caída

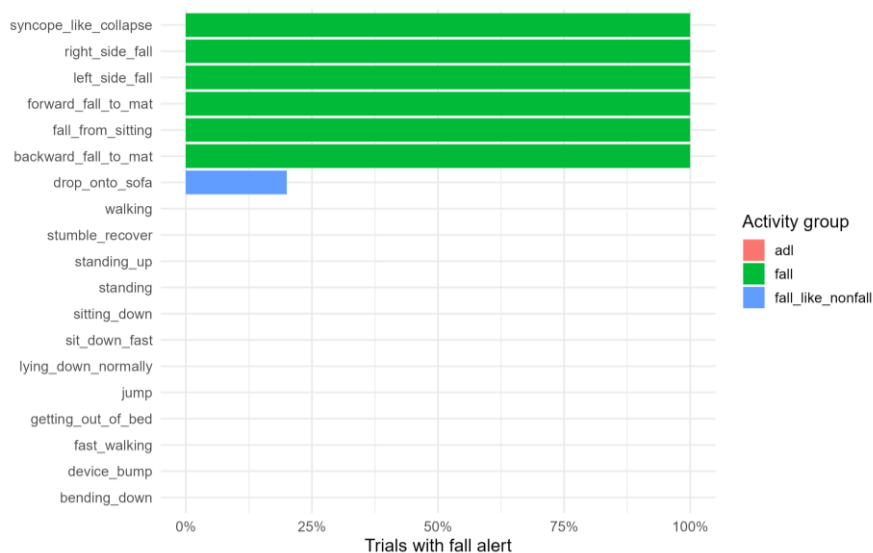


Ilustración 17. Detection Rate by Activity

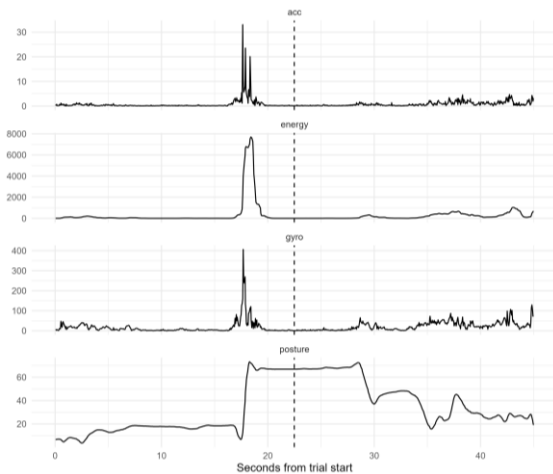


Ilustración 18. Graficas de caída real

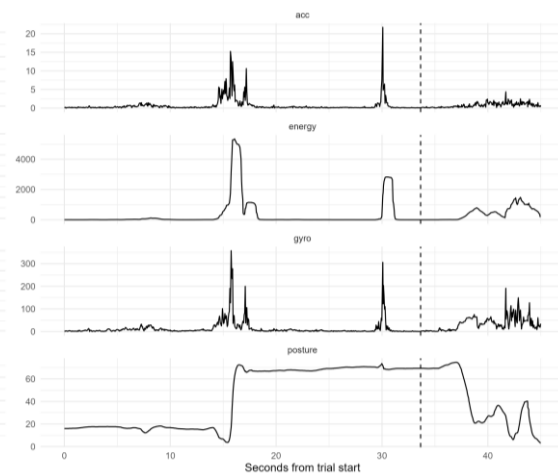


Ilustración 19. Graficos de falso positivo

En las Figuras 18 y 19 se comparan las señales registradas durante una caída real (izquierda) y durante el único falso positivo obtenido (derecha). La caída real presenta un aumento brusco de aceleración, energía y rotación, seguido de una postura tumbada y un periodo de inactividad. El falso positivo, producido al dejarse caer sobre el sofá, muestra un patrón muy similar: impacto inicial, cambio rápido de postura y permanencia en posición tumbada. Aunque los valores son menores que en la caída real, en este caso fueron suficientes como para superar los umbrales que determinan si es una caída.

5.2.2 Pruebas de usabilidad

Se han desarrollado pruebas automáticas para comprobar el comportamiento de la aplicación en escenarios críticos. Estas pruebas se ejecutan sobre el entorno de instrumentación de Android y permiten validar partes del flujo real de la aplicación.

La prueba **CallNoService** valida que la aplicación no se bloquee cuando recibe una alerta de caída y no dispone de permisos para enviar SMS o realizar llamadas. Para ello, la prueba inicia el servicio de detección, conecta la aplicación con un servidor WebSocket simulado, guarda un contacto de emergencia de prueba y envía un mensaje FALL DETECTED. A continuación, espera a que finalice la cuenta atrás y comprueba que la

aplicación llega al flujo de emergencia, pero gestiona correctamente la ausencia de permisos de SMS y llamada.

La prueba ***CancelFalseAlarm*** comprueba que el usuario puede cancelar una alerta de caída antes de que se ejecuten las acciones de emergencia. La prueba inicia el servicio, establece la conexión con el Arduino simulado y envía un mensaje de caída. Después verifica que la aplicación inicia la cuenta atrás y, a continuación, envía la acción de cancelación de alerta. El resultado esperado es que la aplicación registre la cancelación, finalice la alerta y no ejecute ninguna acción posterior, como cargar contactos, enviar SMS o iniciar una llamada. Esta prueba valida el funcionamiento de la cancelación de falsas alarmas y ayuda a justificar el cumplimiento del requisito de permitir al usuario detener una alerta incorrecta.

La prueba ***DobleMensajeCaída*** valida el comportamiento de la aplicación cuando recibe más de un mensaje de caída mientras ya existe una alerta activa. En este escenario, el servidor WebSocket simulado envía un primer mensaje FALL DETECTED, la aplicación inicia la cuenta atrás y, antes de que esta termine, se envía un segundo mensaje de caída. La prueba comprueba que el segundo mensaje se recibe, pero no reinicia la cuenta atrás ni crea una segunda alerta. En su lugar, la aplicación registra que ya existe una alerta activa e ignora el duplicado. Esta prueba es importante porque evita comportamientos repetidos o inconsistentes si el dispositivo envía varios mensajes de caída durante el mismo evento.

La prueba ***MdnsNsd*** valida el descubrimiento automático del dispositivo y la recuperación de la conexión. En primer lugar, registra un servicio NSD simulado con el mismo tipo de servicio utilizado por el Arduino y comprueba que la aplicación lo encuentra, resuelve su dirección y abre una conexión WebSocket correctamente. En segundo lugar, simula una caída de la conexión después del establecimiento inicial del WebSocket y verifica que el servicio de la aplicación detecta la desconexión, programa una reconexión y vuelve a intentar conectarse. Esta prueba justifica el funcionamiento de la conexión automática mediante mDNS/NSD y la robustez de la aplicación ante pérdidas de conexión durante la monitorización.

Capítulo 6

Conclusiones y líneas futuras

6.1 Conclusiones

En este Trabajo de Fin de Grado se ha diseñado, desarrollado y validado un prototipo funcional de dispositivo IoT para la detección de caídas, conectado a una aplicación móvil Android. El sistema integra correctamente el hardware del wearable, el firmware del Arduino, el sensor, la comunicación WIFI/WebSocket y la aplicación móvil encargada de gestionar la monitorización y las alertas de emergencia.

Se puede concluir que se han cumplido los requisitos establecidos para el prototipo. El dispositivo es capaz de inicializarse, leer datos del sensor, ejecutar el algoritmo de detección, comunicarse con la aplicación móvil, enviar alertas en tiempo real y permitir que la aplicación gestione contactos, ubicación, SMS, llamada de emergencia, cancelación de falsas alarmas y registro de logs. Las justificaciones del cumplimiento de los requisitos funcionales y no funcionales se encuentran en el Anexo VII.

Los resultados del estudio estadístico muestran además un comportamiento positivo del algoritmo. Sobre un total de 100 pruebas, solo se generó un falso positivo. Esto dio lugar a una exactitud global del 99,0 %, incluso superior que algunos otros algoritmos de estudios académicos donde se consiguió un 97% [13]. También se obtuvo una tasa de falsos negativos del 0,0 %, lo que indica que ninguna de las caídas simuladas quedó sin detectar durante las pruebas realizadas.

6.2 Líneas futuras

Como líneas futuras, una de las mejoras principales sería incorporar un modelo de inteligencia artificial para complementar el algoritmo actual. Por ejemplo, se podría entrenar un modelo de tipo Random Forest utilizando los datos registrados durante las pruebas, de forma que el sistema pudiera aprender patrones más complejos y mejorar la diferenciación entre caídas reales y actividades normales.

Otra posible mejora sería añadir un botón físico al dispositivo wearable. Este botón permitiría al usuario activar manualmente una alerta en caso de que el sistema no detectase una caída real, reduciendo así el riesgo asociado a falsos negativos. También podría utilizarse como mecanismo adicional de seguridad en situaciones en las que el usuario necesite ayuda, aunque no se haya producido una caída.

Por último, sería recomendable mejorar la interfaz y la experiencia de usuario de la aplicación móvil. Aunque la aplicación cumple las funciones principales del sistema, futuras versiones podrían optimizar la navegación, simplificar el proceso de configuración, mejorar la presentación de alertas y adaptar mejor el diseño a usuarios mayores o con poca experiencia tecnológica.

Bibliografía

[1] Organización Mundial de la Salud. (2021, 26 de abril). Falls. <https://www.who.int/news-room/fact-sheets/detail/falls>

[2] Bosch Sensortec. (s. f.). BNO055 intelligent 9-axis absolute orientation sensor [Datasheet]. Recuperado el 2 de marzo de 2026, de <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bno055-ds000.pdf>

[3] Bosch Sensortec. (s. f.). BNO055 quick start guide. Recuperado el 2 de marzo de 2026, de https://www.bosch-sensortec.com/media/boschsensortec/downloads/application_notes_1/bst-bno055-an007.pdf

[4] Townsend, K. (2025, mayo). Adafruit BNO055 absolute orientation sensor. Adafruit Learning System. Recuperado el 10 de marzo de 2026, de <https://cdn-learn.adafruit.com/downloads/pdf/adafruit-bno055-absolute-orientation-sensor.pdf>

[5] Adafruit. (s. f.). Adafruit BNO055 absolute orientation sensor. Recuperado el 10 de marzo de 2026, de <https://learn.adafruit.com/adafruit-bno055-absolute-orientation-sensor?view=all>

[6] Adafruit. (s. f.). Arduino code. En Adafruit BNO055 absolute orientation sensor. Recuperado el 20 de marzo de 2026, de <https://learn.adafruit.com/adafruit-bno055-absolute-orientation-sensor/arduino-code>

[7] Adafruit. (s. f.). Device calibration. En Adafruit BNO055 absolute orientation sensor. Recuperado el 20 de marzo de 2026, de <https://learn.adafruit.com/adafruit-bno055-absolute-orientation-sensor/device-calibration>

[8] Adafruit. (s. f.). Adafruit_BNO055 [Repositorio de GitHub]. GitHub. Recuperado el 20 de marzo de 2026, de https://github.com/adafruit/Adafruit_BNO055

- [9] Adafruit. (s. f.). Adafruit_BNO055.cpp [Código fuente]. GitHub. Recuperado el 11 de Abril de 2026, de https://github.com/adafruit/Adafruit_BNO055/blob/master/Adafruit_BNO055.cpp
- [10] Huynh, Q. T., Nguyen, U. D., Irazabal, L. B., Ghassemian, N., & Tran, B. Q. (2015). Optimization of an accelerometer and gyroscope-based fall detection algorithm. *Journal of Sensors*, 2015, Article 452078. <https://doi.org/10.1155/2015/452078>
- [11] Li, Q., Stankovic, J. A., Hanson, M. A., Barth, A. T., Lach, J., & Zhou, G. (2009). Accurate, fast fall detection using gyroscopes and accelerometer-derived posture information. Recuperado el 15 de abril de 2026, de <https://www.cs.virginia.edu/~stankovic/psfiles/bsn09-1.pdf>
- [12] Ahn, S., Kim, Y., Kim, H., Koo, C., & Kim, D. (2019). Evaluation of inertial sensor-based pre-impact fall detection algorithms using public dataset. Recuperado el 15 de abril de 2026, de <https://pmc.ncbi.nlm.nih.gov/articles/PMC6412321/>
- [13] Sucerquia, A., López, J. D., & Vargas-Bonilla, J. F. (2016). Two-threshold energy based fall detection using a triaxial accelerometer. En *Proceedings of the Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*. Recuperado el 18 de abril de 2026, de <https://pubmed.ncbi.nlm.nih.gov/28268967/>
- [14] Kambhampati, S. S., Singh, V., Manikandan, M. S., & Ramkumar, B. (2015). Unified framework for triaxial accelerometer-based fall event detection and classification using cumulants and hierarchical decision tree classifier. *Healthcare Technology Letters*, 2(4), 101–107. <https://doi.org/10.1049/htl.2015.0018>
- [15] Chen, M., Wang, H., Yu, L., Yeung, E. H. K., Luo, J., Tsui, K. L., & Zhao, Y. (2022). A systematic review of wearable sensor-based technologies for fall risk assessment in older adults. *Sensors*, 22(18), Article 6752. <https://doi.org/10.3390/s22186752>
- [16] Bagalà, F., Becker, C., Cappello, A., Chiari, L., Aminian, K., Hausdorff, J. M., Zijlstra, W., & Klenk, J. (2012). Evaluation of accelerometer-based fall detection algorithms on

real-world falls. PLOS ONE, 7(5), Article e37062.
<https://doi.org/10.1371/journal.pone.0037062>

[17] Hsiao, E. T., & Robinovitch, S. N. (2008). Effect of pre-impact movement strategies on the impact forces resulting from a lateral fall. Recuperado el 15 de mayo de 2026, de <https://pmc.ncbi.nlm.nih.gov/articles/PMC2562433/>

[18] Yang, X., & Lockhart, T. (2014). Trunk angular kinematics during slip-induced backward falls and activities of daily living. Recuperado el 30 de abril de 2026, de <https://pmc.ncbi.nlm.nih.gov/articles/PMC4127473/>

[19] Tong, Y., Jiang, J. Y., Chen, Y. L., & Xue, S. H. (2012). Exploration and implementation of a pre-impact fall recognition method based on an inertial body sensor network. Recuperado el 15 de mayo de 2026, de <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3522966/>

[20] Bruijns, D. A., & Smeets, M. J. (2005). Review of motor control mechanisms underlying impact absorption from falls. Recuperado el 16 de mayo de 2026, de <https://www.sciencedirect.com/science/article/abs/pii/S0966636204000207>

[21] Pan, D., & Liu, H. (2021). Human falling recognition based on movement energy expenditure feature. Recuperado el 15 de abril de 2026, de <https://onlinelibrary.wiley.com/doi/10.1155/2021/1422586>

[22] Elbasiony, M., Elhoseny, M. A., Elkazaz, A., & Mahmoud, A. M. (2021). An IoT-based framework for remote fall monitoring. arXiv. Recuperado el 12 de mayo de 2026, de <https://arxiv.org/abs/2105.09461>

[23] Newaz, N. T., et al. (2023). The methods of fall detection: A literature review. *Sensors*, 23(11), Article 5212. <https://doi.org/10.3390/s23115212>

[24] Bourke, A. K., O'Brien, J. V., & Lyons, G. M. (2007). Evaluation of a threshold-based tri-axial accelerometer fall detection algorithm. *Gait & Posture*, 26(2), 194–199. <https://doi.org/10.1016/j.gaitpost.2006.09.012>

- [25] Noury, N., Fleury, A., Rumeau, P., Bourke, A. K., Laighin, G. Ó., Rialle, V., & Lundy, J. E. (2007). Fall detection—principles and methods. En *Proceedings of the Annual International Conference of the IEEE Engineering in Medicine and Biology Society* (pp. 1663–1666). <https://doi.org/10.1109/IEMBS.2007.4352627>
- [26] Brown, R. G., & Hwang, P. Y. C. (2012). *Introduction to random signals and applied Kalman filtering with MATLAB exercises* (4.^a ed.). John Wiley & Sons.
- [27] A systematic review of fall detection and prediction technologies for older adults: An analysis of sensor modalities and computational models. (2026). *Applied Sciences*, 16(4), Article 1929. <https://www.mdpi.com/2076-3417/16/4/1929>
- [28] A decade of progress in wearable sensors for fall detection (2015–2024): A network-based visualization review. (2025). *Sensors*, 25(7), Article 2205. <https://www.mdpi.com/1424-8220/25/7/2205>
- [29] Toward real-time, robust wearable sensor fall detection using deep learning methods: A feasibility study. (2025). *Sensors*, 25(17), Article 5249. <https://www.mdpi.com/1424-8220/25/17/5249>
- [30] Cross-dataset evaluation of wearable fall detection systems using data from real falls and long-term monitoring of daily life. (2024). Recuperado el 29 de mayo de 2026, de <https://www.sciencedirect.com/science/article/pii/S0263224124008777>
- [31] A study of the use of gyroscope measurements in wearable fall detection systems. (2020). *Symmetry*, 12(4), Article 649. <https://www.mdpi.com/2073-8994/12/4/649>
- [32] Machine learning-based framework for pre-impact same-level fall detection using IMU-derived features. (2025). *Biosensors*, 15(9), Article 618. <https://www.mdpi.com/2079-6374/15/9/618>
- [33] A comparative overview of technological advances in fall detection systems for elderly people. (2025). *Sensors*, 25(24), Article 7423. <https://www.mdpi.com/1424-8220/25/24/7423>

- [34] State-of-the-art review on fall prediction among older adults: Exploring edge devices as a promising approach for the future. (2025). Recuperado el 10 de marzo de 2026, de <https://www.sciencedirect.com/science/article/pii/S2665917425000728>
- [35] Android Developers. (s. f.). Foreground services overview. Recuperado el 24 de mayo de 2026, de <https://developer.android.com/develop/background-work/services/fgs>
- [36] Android Developers. (s. f.). Foreground service types. Recuperado en mayo de 2026, de <https://developer.android.com/develop/background-work/services/fgs/service-types>
- [37] Android Developers. (s. f.). Request runtime permissions. Recuperado en mayo de 2026, de <https://developer.android.com/training/permissions/requesting>
- [38] Android Developers. (s. f.). Access location in the background. Recuperado en mayo de 2026, de <https://developer.android.com/develop/sensors-and-location/location/background>
- [39] Android Developers. (s. f.). LocationManager. Recuperado en mayo de 2026, de <https://developer.android.com/reference/kotlin/android/location/LocationManager>
- [40] Android Developers. (s. f.). Common intents. Recuperado en junio de 2026, de <https://developer.android.com/guide/components/intents-common>
- [41] Android Developers. (s. f.). Background activity launch restrictions. Recuperado en mayo de 2026, de <https://developer.android.com/guide/components/activities/background-starts>
- [42] Android Developers. (s. f.). Companion device pairing. Recuperado en abril de 2026, de <https://developer.android.com/develop/connectivity/bluetooth/companion-device-pairing>

[43] Android Developers. (s. f.). Wi-Fi Aware overview. Recuperado en mayo de 2026, de <https://developer.android.com/develop/connectivity/WIFI/WIFI-aware>

[44] Google Play Console Help. (s. f.). Use of SMS or Call Log permission groups. Recuperado en mayo de 2026, de <https://support.google.com/googleplay/android-developer/answer/10208820?hl=en>

[45] Open Source Initiative. (s. f.). GNU General Public License version 3. Recuperado en junio de 2026, de <https://opensource.org/license/gpl-3-0>

[46] Arduino. (s. f.). Nano ESP32. Recuperado en marzo de 2026, de <https://docs.arduino.cc/nano-esp32>

[47] Arduino. (2025, 19 de febrero). Arduino Nano ESP32 user manual. Recuperado en abril de 2026, de <https://docs.arduino.cc/tutorials/nano-esp32/cheat-sheet/>

[48] Arduino. (2024, 24 de octubre). Getting started with Nano ESP32. Recuperado en abril de 2026, de <https://docs.arduino.cc/tutorials/nano-esp32/getting-started-nano-esp32>

[49] Adafruit. (s. f.). PowerBoost 500 Charger—Rechargeable 5V Lipo USB boost @ 500mA+. Recuperado en junio de 2026, de <https://www.adafruit.com/product/1944>

[50] Adafruit. (2024, 3 de junio). Adafruit PowerBoost 500 + Charger [PDF]. Recuperado en junio de 2026, de <https://cdn-learn.adafruit.com/downloads/pdf/adafruit-powerboost-500-plus-charger.pdf>

[51] Android Developers. (s. f.). Kotlin and Android. Recuperado en abril de 2026, de <https://developer.android.com/kotlin>

[52] Android Developers. (2026, 6 de marzo). Kotlin overview. Recuperado en abril de 2026, de <https://developer.android.com/kotlin/overview>

[53] Android Developers. (2024, 3 de enero). Use network service discovery. Recuperado en mayo de 2026, de <https://developer.android.com/develop/connectivity/WIFI/use-nsd>

[54] Android Developers. (2025, 20 de agosto). android.net.nsd. Recuperado en mayo de 2026, de <https://developer.android.com/reference/android/net/nsd/package-summary>

[55] Square. (s. f.). OkHttp. Recuperado en mayo de 2026, de <https://square.github.io/okhttp/>

[56] Square. (s. f.). WebSocket. En OkHttp 5.x API. Recuperado en mayo de 2026, de <https://square.github.io/okhttp/5.x/okhttp/okhttp3/-web-socket/index.html>

[57] Espressif Systems. (s. f.). Welcome to ESP32 Arduino Core's documentation. Recuperado en marzo de 2026, de <https://docs.espressif.com/projects/arduino-esp32/>

[58] Espressif Systems. (s. f.). Wi-Fi API. En Arduino ESP32 documentation. Recuperado en marzo de 2026, de <https://docs.espressif.com/projects/arduino-esp32/en/latest/api/WIFI.html>

[59] Espressif Systems. (s. f.). arduino-esp32 [Repositorio de GitHub]. GitHub. Recuperado en junio de 2026, de <https://github.com/espressif/arduino-esp32>

[60] Espressif Systems. (s. f.). ESPmDNS.h [Código fuente]. GitHub. Recuperado en mayo de 2026, de <https://github.com/espressif/arduino-esp32/blob/master/libraries/ESPMDNS/src/ESPMDNS.h>

[61] Espressif Systems. (s. f.). mDNS service. En ESP-IDF Programming Guide. Recuperado en abril de 2026, de <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/mdns.html>

[62] Sattler, M. (s. f.). arduinoWebSockets [Repositorio de GitHub]. GitHub. Recuperado en mayo de 2026, de <https://github.com/Links2004/arduinoWebSockets>

[63] Arduino Libraries. (s. f.). WebSockets. Recuperado en mayo de 2026, de <https://www.arduino-libraries.info/libraries/web-sockets>

[63] GLOBAL SMARTPHONE OS MARKET SHARE, *Counterpointresearch.com*, 2024. Recuperado en junio de 2026, <https://counterpointresearch.com/en/insights/global-smartphone-os-market-share>

Uso de herramientas de inteligencia artificial generativa

La inteligencia artificial se ha usado en este proyecto como herramienta de apoyo. Su uso principal ha sido de ayuda a la investigación, para la búsqueda de fuentes de información, sobre todo de tipo estudio académico.

Su uso también ha sido de apoyo a la resolución de problemas, ambos de la lógica del proyecto como de la implementación a la hora de solucionar problemas en el código. Su mayor peso viene de la ayuda para la creación del script de estudio estadístico para la verificación de la precisión del algoritmo.

Por último, se ha usado de manera puntual para corregir, revisar y mejorar ciertos párrafos de la memoria con la intención de que el contenido fuera más claro y coherente.

No se ha usado la inteligencia artificial en la toma de decisiones de fases críticas del diseño o implementación.

Anexo I – Librerías del proyecto

Librerías del firmware del Arduino:

Librería	Función en el proyecto
WIFI.h	Gestiona la conectividad WIFI del ESP32. Se usa tanto para conectarse a una red guardada como para crear el punto de acceso temporal durante el aprovisionamiento.
WebServer.h	Implementa el servidor HTTP usado en el modo setup. Permite responder a rutas como /status, /provision y /reset.
WebSocketsServer_Generic.h	Implementa el servidor WebSocket del Arduino en el puerto 81. Se utiliza para enviar mensajes JSON a la aplicación Android en tiempo real.
ESPmDNS.h	Permite anunciar el dispositivo en la red local mediante mDNS. Gracias a esta librería, el Arduino puede publicarse como fall-detector.local y anunciar el servicio _fallws._tcp.
Preferences.h	Permite guardar datos en la memoria no volátil del ESP32. Se utiliza para almacenar credenciales WIFI y datos de calibración del BNO055.
Wire.h	Gestiona la comunicación I2C entre el ESP32 y el sensor BNO055.
Adafruit_Sensor.h	Proporciona una interfaz común para sensores de Adafruit. Sirve como base para trabajar con el BNO055.

Adafruit_BNO055.h	Librería principal para comunicarse con el sensor BNO055. Permite obtener aceleración lineal, giroscopio, orientación Euler, vector de gravedad y datos de calibración.
utility/imumaths.h	Proporciona estructuras matemáticas para manejar vectores y cálculos relacionados con la IMU, como los vectores de aceleración, giroscopio y gravedad.

Tabla 4. Librerías del Arduino

Librerías de la app:

Librería / API	Función en el proyecto
Kotlin	Lenguaje principal de desarrollo de la aplicación Android. Se utiliza para implementar la interfaz, el servicio de monitorización, la lógica de conexión, almacenamiento y emergencia.
Jetpack Compose	Framework usado para construir la interfaz gráfica de la app de forma declarativa. Permite crear las pantallas de monitorización, configuración del dispositivo, contactos y alerta de caída.
Material 3	Biblioteca de componentes visuales utilizada junto con Jetpack Compose para crear una interfaz moderna y consistente: botones, tarjetas, diálogos, campos de texto, etc.
OkHttp	Librería de red utilizada para las comunicaciones HTTP y WebSocket. En el proyecto se usa para enviar peticiones de aprovisionamiento al Arduino y para abrir la conexión WebSocket con el dispositivo.

Android NSD / NsdManager	API de Android para descubrimiento de servicios en red local mediante DNS-SD/mDNS. Se usa para buscar el servicio _fallws._tcp anunciado por el Arduino.
WiFiManager.MulticastLock	API utilizada para permitir la recepción de tráfico multicast necesario para el descubrimiento mDNS/NSD en algunas redes WIFI.
Android Foreground Service API	Permite que FallDetectionService siga ejecutándose en segundo plano mientras mantiene la conexión WebSocket con el Arduino.
Android Notification API	Se usa para mostrar la notificación persistente del servicio de monitorización y actualizar el estado de conexión o alerta.
Android Broadcast API	Permite que FallDetectionService comunique eventos a MainActivity, como logs, inicio de alerta, cuenta atrás y finalización de alerta.
SharedPreferences	API de almacenamiento local usada para guardar datos sencillos como la dirección del dispositivo, la opción de autoconexión y contactos/configuración según el caso.
org.json	Librería usada para crear, leer y procesar mensajes JSON recibidos desde el Arduino o almacenados en logs.
Android Location API	Se utiliza para obtener la ubicación del teléfono durante una emergencia y generar un enlace de Google Maps.

SmsManager	API de telefonía usada para enviar SMS automáticos a todos los contactos de emergencia cuando se confirma una caída y no se cancela la alerta.
Android Telecom / Llamadas	API usada para iniciar una llamada telefónica al primer contacto de emergencia configurado.
Vibrator / VibrationEffect	API usada para hacer vibrar el teléfono durante la cuenta atrás de alerta de caída.
ToneGenerator	API usada para emitir pitidos periódicos durante la alerta de caída.
FileProvider	Permite compartir de forma segura los archivos de log generados por la aplicación.

Tabla 5. Librerías de Android Studio

Anexo II – Diseño de la caja

Caja Interna:

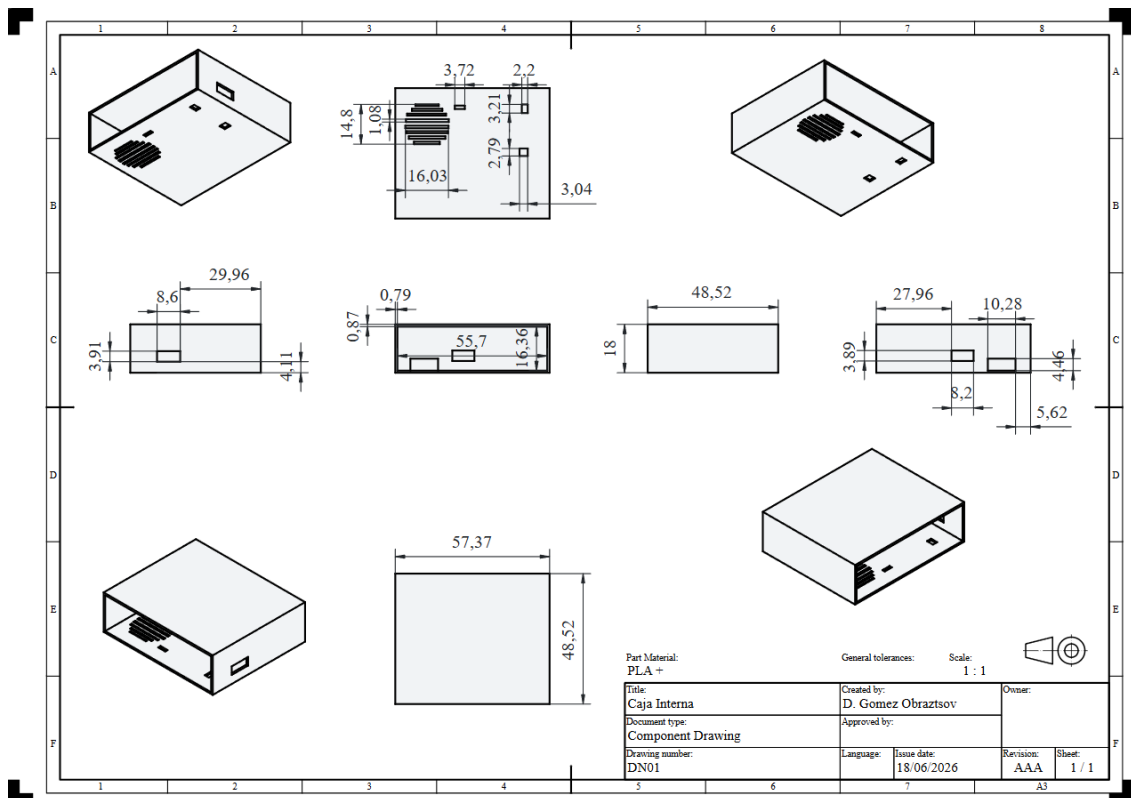


Ilustración 20. Esquema de caja interna.

Caja externa:

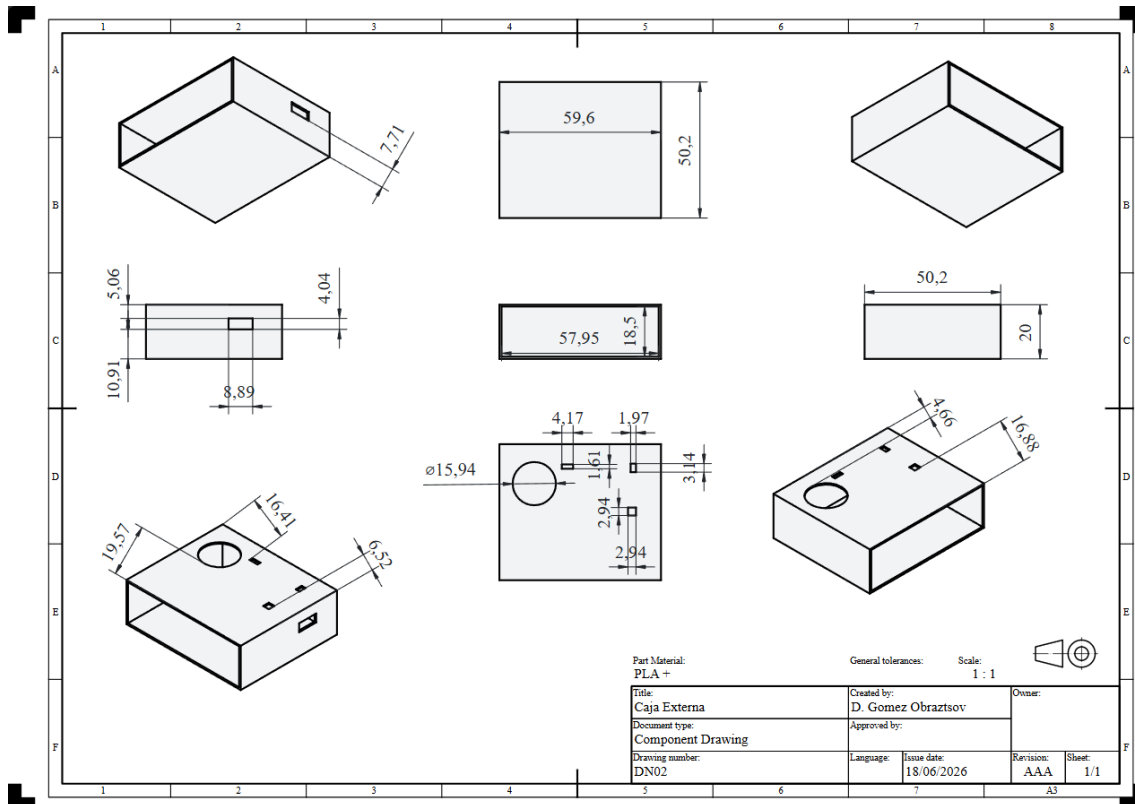


Ilustración 21. Esquema de caja externa.

Estos son los diseños finales de las cajas donde se guardan los componentes electrónicos que componen el dispositivo de detección de caídas. Diseñadas en FreeCAD e impresas con una impresora 3D usando el material PLA +.

Originalmente, antes de hacer el cambio a un custom PCB, el diseño de la caja era diferente. Los diseños se encuentran en el Anexo.

La pinza usada para sujetar el dispositivo al cuerpo del usuario se seleccionó de la página *Thingiverse*, hecha por el usuario "pmagda".
<https://www.thingiverse.com/thing:325983/files>

Diseño Anterior:

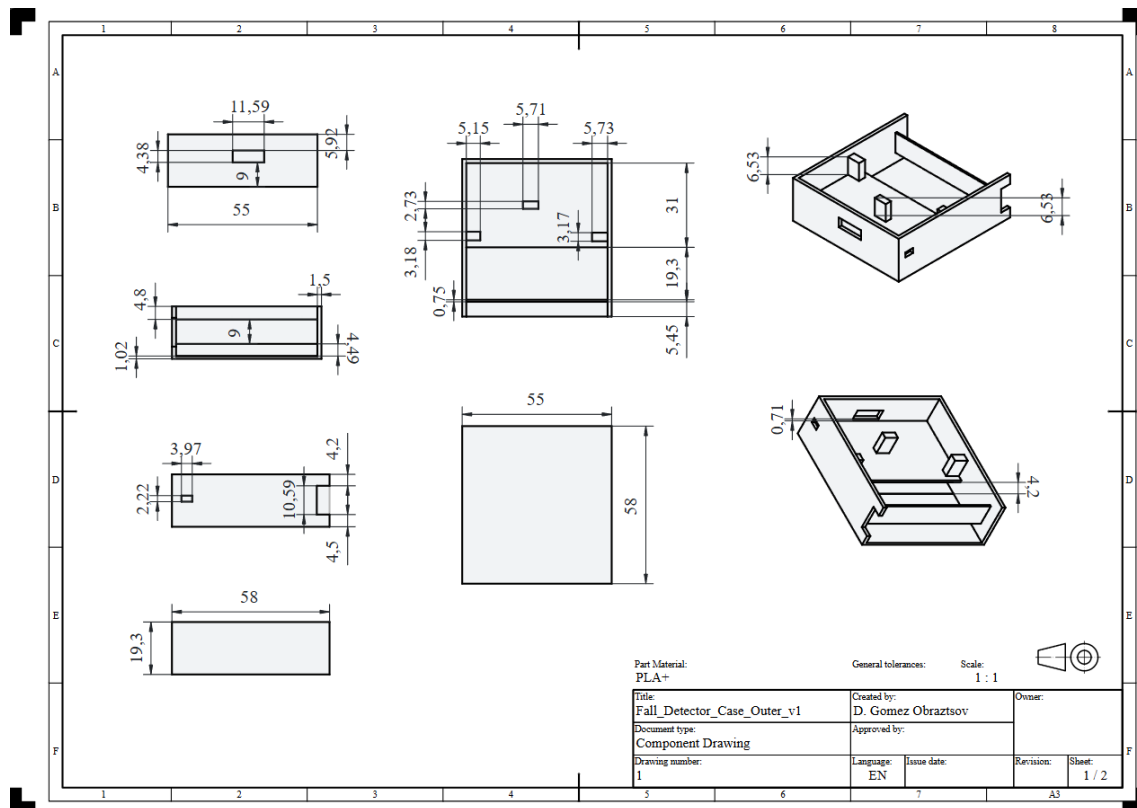
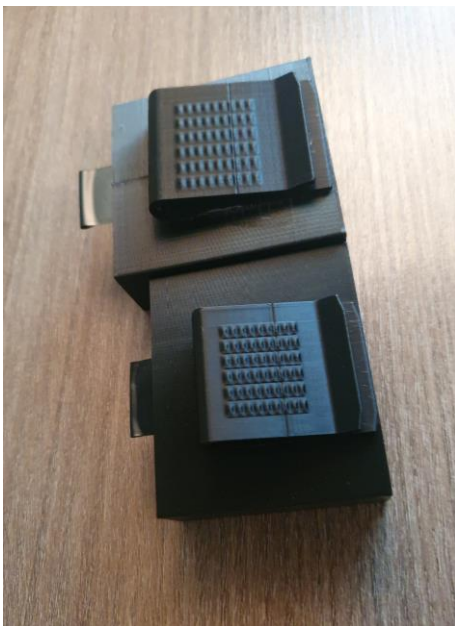


Ilustración 22. Esquema de caja interna (diseño anterior).

Comparativa de tamaño:



Ilustración 23. Comparacion de tamaño



Anexo III – Conexiones del hardware

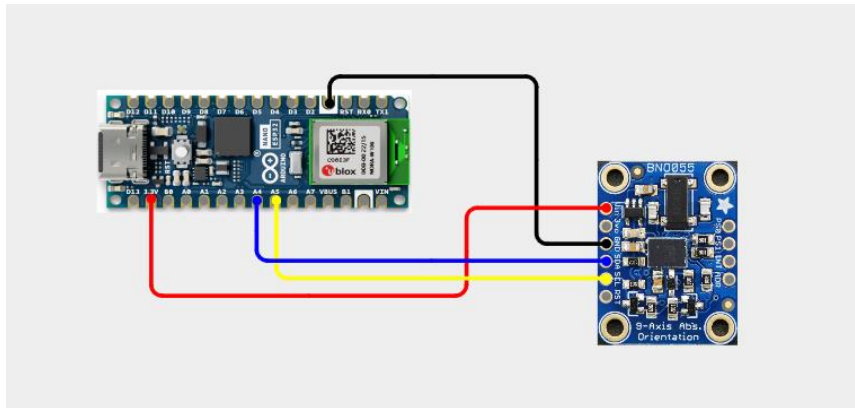


Ilustración 24. Conexión entre Arduino y sensor BNO055

BNO-055 (PIN)	Arduino Nano ESP32 (PIN)	Notas
VIN	3.3V	Power input
GND	GND	Toma de tierra
SDA	A4	I2C Data line
SCL	A5	I2C Clock line

Tabla 6. Conexiones arduino-sensor

VIN (Power Input)

Toma de corriente para el sensor

El Pin VIN tiene que estar conectado al pin de 3.3V (o 5V, depende del modelo específico) del Arduino.

GND (Ground)

Toma de tierra, para referencia eléctrica para ambos componentes.

El pin de GND garantiza que ambos tengan la misma referencia de voltaje. Necesario para garantizar la comunicación y que no haya diferencias de voltaje que puedan dañar los componentes.

SDA (Serial Data Line)

Línea de transmisión de datos entre el sensor y el Arduino.

El pin SDA es parte del protocolo I2C que se usa para enviar los datos entre ambos componentes. Es bidireccional. Se conecta a un pin analógico del Arduino, ya que puede tomar muchos valores (no sólo binario como los digitales).

SCL (Serial Clock Line)

Da timing para la transmisión de datos

El pin SCL es la línea de reloj para el protocolo I2C. El Arduino manda una señal en intervalos regulares, coordinando cuando se tiene que leer o escribir los datos que pasan por la línea de SDA.

Circuito de carga

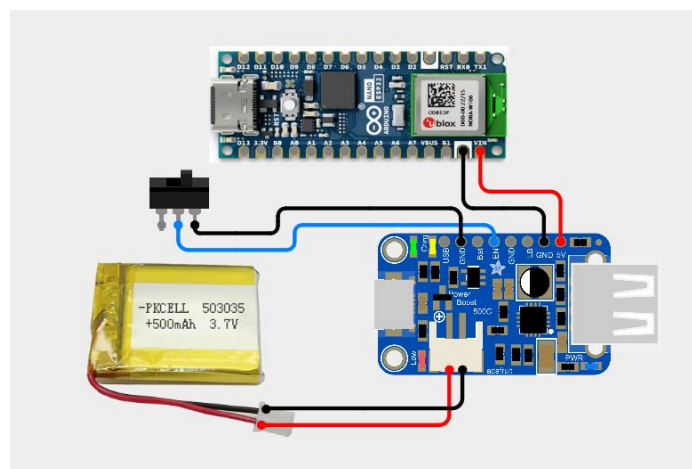


Ilustración 25. Circuito de carga

Unión del módulo de carga y un step-up voltaje.

- La batería elegida es una Li-Po de 3.7V 500mAh.
- El módulo de carga y boost es el PowerBoost 500C

DEL PIN:	AL PIN:
BAT+	PowerBoost JST VBAT
BAT-	PowerBoost JST GND
PowerBoost 5V	ARDUINO Vin
PowerBoost GND	ARDUINO GND
Switch Common	PowerBoost EN
Switch L1/L2	GND

Tabla 7. Conexiones circuito carga

Esto convierte la carga de la batería de 3.7V a una carga de 5V que puede encender el Arduino desde su pin VIN.

Anexo IV – Dispositivo wearable

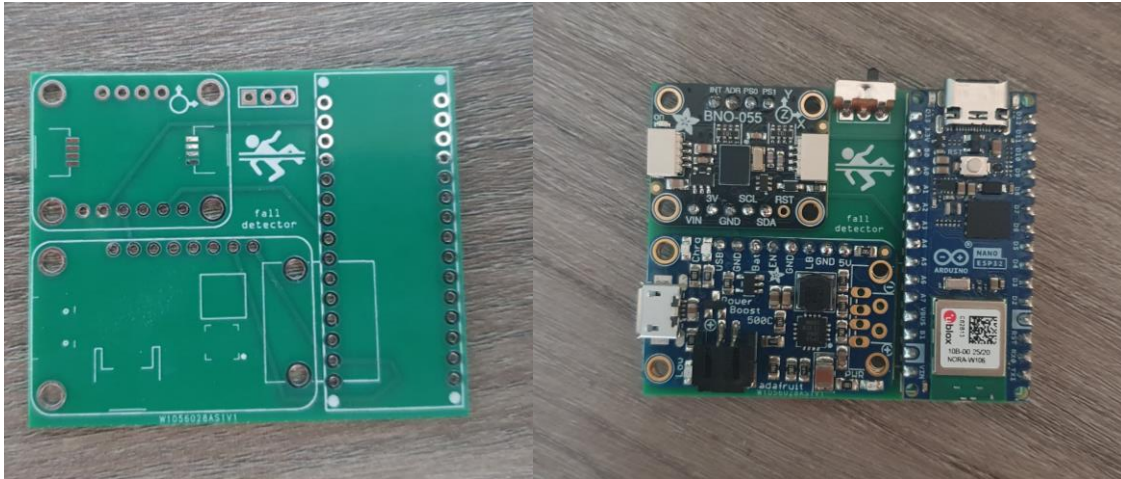


Ilustración 26. Custom PCB



Ilustración 27. PCB dentro de la caja.

Anexo V – Wireframes y resultados

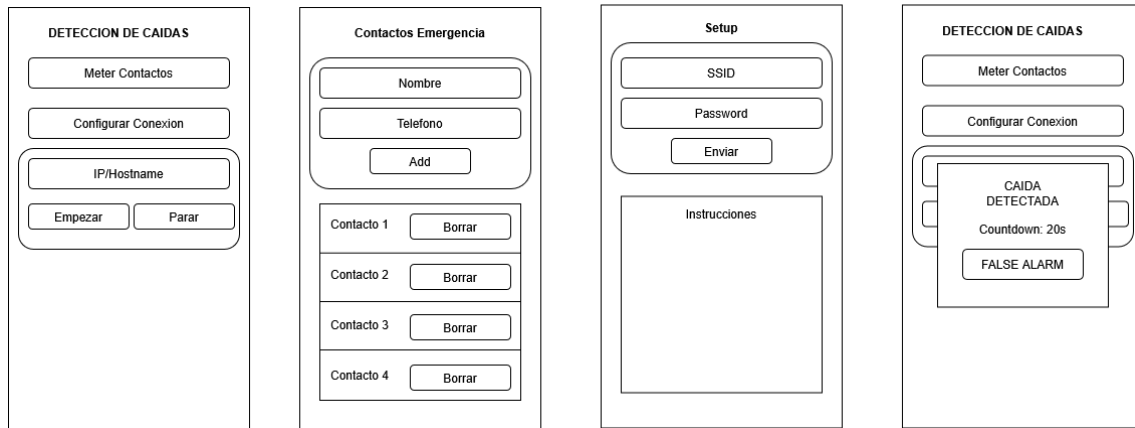


Ilustración 28. Wireframes de la aplicación móvil

Resultados

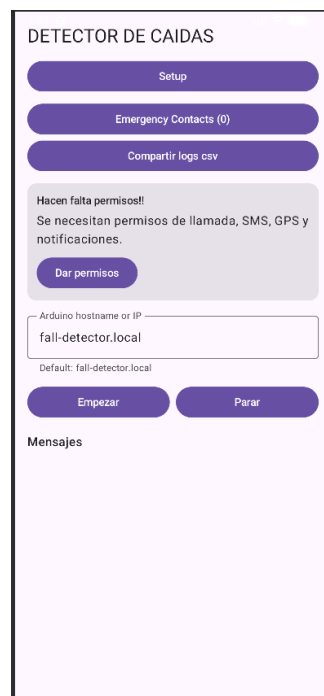
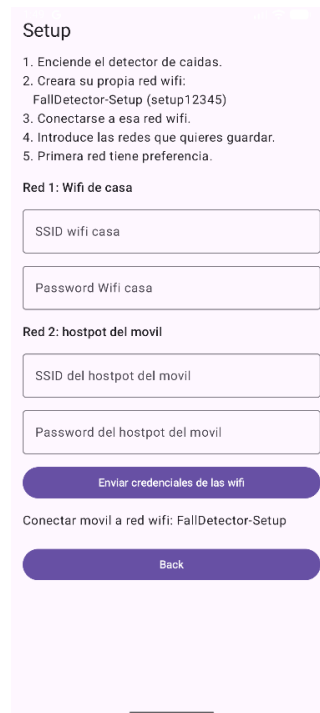


Ilustración 29. Pagina Principal



Setup

1. Enciende el detector de caídas.
2. Crea su propia red wifi:
FallDetector-Setup (setup12345)
3. Conectarse a esa red wifi.
4. Introduce las redes que quieres guardar.
5. Primera red tiene preferencia.

Red 1: Wifi de casa

SSID wifi casa

Password Wifi casa

Red 2: hotspot del movil

SSID del hotspot del movil

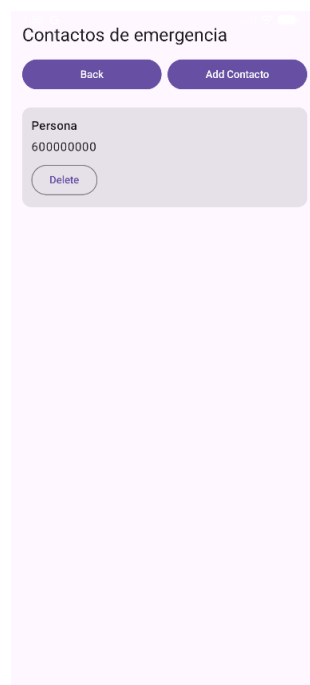
Password del hotspot del movil

Enviar credenciales de las wifi

Conectar movil a red wifi: FallDetector-Setup

Back

Ilustración 30. Página de setup



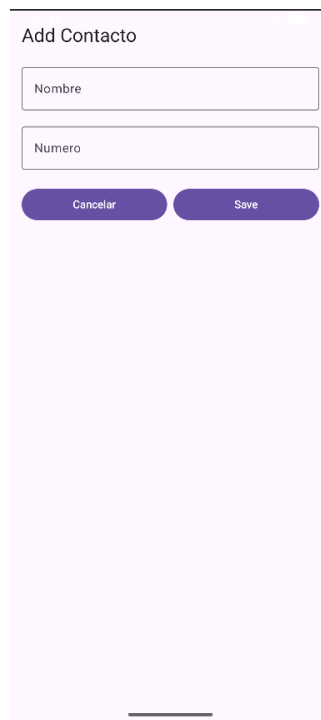
Contactos de emergencia

Back Add Contacto

Persona
600000000

Delete

Ilustración 31. Página de Contactos



Add Contacto

Nombre

Numero

Cancelar Save

Ilustración 32. Página de añadir contacto

Anexo VI – Repositorio del proyecto

El código fuente del sistema desarrollado se puede encontrar en el repositorio del proyecto: https://github.com/danielgomezobraztsov/tfg-fall_detection_device

En ello, además del código fuente de la aplicación HMI del dispositivo móvil, se pueden encontrar toda la información necesaria para el desarrollo hardware necesario del sistema. Para poder desplegar este proyecto haría falta obtener el hardware necesario, subir el firmware del Arduino Nano ESP32 (/algorithm/final_version/final_version.ino) e instalar la aplicación móvil en un dispositivo Android.

Anexo VII – Justificación de Requisitos

Requisitos Funcionales:

(RF1): Al encender el dispositivo usando el interruptor del PCB, el dispositivo se inicializa en *setup()*, donde se inicia la comunicación, se inicializa el sensor BNO055 mediante *bno.begin()* y se restauran la calibración anterior (si existe). Después, el sistema inicializa los servicios de comunicación necesarios según el modo: servidor HTTP en *setup* o WebSocket/mDNS en modo normal.

(RF2): El arduino intenta conectarse a redes WIFI guardadas usando *connectToSavedWIFI()*. Si no existen credenciales válidas o ninguna red responde, se ejecuta *startSetupMode()*, entrando automáticamente en modo de aprovisionamiento.

(RF3): En modo *setup*, el Arduino crea un hotspot WIFI con *WIFI.mode(WIFI_AP)* y *WIFI.softAP()*. La red creada es FallDetector-Setup, permitiendo que el móvil se conecte directamente al dispositivo.

(RF4): La app permite introducir credenciales WIFI desde la pantalla de configuración del dispositivo. Estas credenciales se envían al Arduino con *ProvisioningClient.provisionWIFI()*, que realiza una petición POST a <http://192.168.4.1/provision>.

(RF5): El sistema permite almacenar hasta cinco redes WIFI (*MAX_WIFI_CREDENTIALS = 5*). La prioridad queda determinada por el orden de guardado, se intenta conectarse primero a la red almacenada en la posición 0. Actúa como una FIFO, cuando se añade una nueva cuando hay 5 redes ya guardadas, se borra la primera en haberse metido.

(RF6): Las credenciales WIFI se almacenan en memoria no volátil usando Preferences, dentro del namespace "WIFI". Se guardan campos como count, s0, p0, s1, p1, ... Asi permanecen disponibles tras reiniciar el Arduino.

(RF7): Después de recibir correctamente las credenciales en */provision*, el Arduino las guarda con *saveWIFICredentials()* y ejecuta un reinicio con *ESP.restart()*. Al arrancar de nuevo, vuelve a intentar la conexión usando las redes almacenadas.

(RF8): En modo normal, el Arduino se conecta a una de las redes WIFI guardadas mediante *connectToSavedWiFi()* y *tryConnectToWIFICredential()*. Si la conexión se realiza correctamente, se ejecuta *startNormalMode()*. Donde empieza el modo normal de monitorización.

(RF9): El dispositivo se anuncia en la red local mediante mDNS en *startMdnsService()*. Publica el hostname *fall-detector.local*, el servicio *_fallws._tcp.local* y el puerto WebSocket 81.

(RF10): La aplicación permite introducir manualmente un hostname, dirección IP o URL WebSocket en la pantalla principal. La dirección se guarda usando *DeviceConfigStorage* y se usa posteriormente para hacer el endpoint de conexión.

(RF11): La conexión WebSocket se establece desde *FallDetectionService* usando OkHttp. El endpoint transforma el host o IP introducido en URL con el formato *ws://<host>:81/*.

(RF12): El usuario puede iniciar la monitorización desde la app pulsando el botón de inicio. Esta acción envía un intent con *ACTION_CONNECT* a *FallDetectionService*, que comienza el proceso de conexión con el Arduino.

(RF13): El usuario puede detener la monitorización desde la app mediante el botón de parar. Se envía *ACTION_DISCONNECT*, cierra el WebSocket, marca *userDisconnected = true* y detiene el servicio de monitorización.

(RF14): El Arduino adquiere periódicamente datos del BNO055 en *runFallDetection()*. En cada ciclo lee aceleración lineal, giroscopio, orientación Euler y vector de gravedad mediante los vectores *VECTOR_LINEARACCEL*, *VECTOR_GYROSCOPE*, *VECTOR_EULER* y *VECTOR_GRAVITY*.

(RF15): Se calcula las principales métricas de detección: *accMag*, *gyroMag*, *fallEnergy*, *totalEnergy* y *postureAngle*. Además, actualiza los estados de postura tumbada e inactividad necesarios para confirmar una caída.

(RF16): La detección de una posible caída se realiza cuando se cumplen los umbrales *totalEnergy* > *ENERGY_THRESHOLD* y *gyroMag* > *GYRO_THRESHOLD*. En el código, estos valores están a *ENERGY_THRESHOLD* = 250 y *GYRO_THRESHOLD* = 150.

(RF17): Cuando se detecta una caída candidata, se inicia la captura de una event window en *startEventCapture()*. La ventana incluye muestras previas y posteriores al trigger, con 38 muestras antes y 38 después, un total de 76 muestras.

(RF18): La caída no se confirma solo por el impacto inicial. Se exige que, dentro de *FALL_CANDIDATE_TIMEOUT_MS* = 10000 (ms), el dispositivo esté en postura tumbada e inactivo durante *inactivityTime* = 3000 (ms). Y si se cumple, se envía la alerta de caída.

(RF19): El Arduino envía mensajes en tiempo real usando WebSocket con las funciones *sendMessage()* y *websocket.broadcastTXT()*. Los tipos de mensaje que se usan son status, debug, event, event_window y fall.

(RF20): La app muestra en vivo los mensajes recibidos del Arduino mediante los broadcasts enviados desde *FallDetectionService* hacia *MainActivity*. La interfaz presenta estados de conexión, mensajes de depuración y eventos relacionados con la detección.

(RF21): La app guarda localmente los mensajes y eventos usando *DataLogger*. Se generan logs como *arduino_messages_raw.jsonl*, *arduino_messages.csv*, *arduino_debug.csv*, *arduino_event_samples.csv* y *app_events.csv*. Con estos archivos es como se ha hecho el estudio estadístico de pruebas.

(RF22): La app permite añadir, visualizar y eliminar contactos de emergencia en la interfaz de *MainActivity*. Los contactos se representan con la clase *EmergencyContact* y se guardan/cargan correctamente con *EmergencyContactStorage*, para manter nombre y número de teléfono de forma persistente.

(RF23): La app solicita los permisos necesarios para su funcionamiento. Esto incluye permisos de SMS, llamadas, notificaciones, ubicación, vibración, internet y descubrimiento de red local.

(RF24): La recepción de alertas de caída se gestiona en *FallDetectionService.handleEsp32Message()*. Si el mensaje recibido desde el Arduino tiene tipo *fall* o contiene el texto *FALL DETECTED*, se ejecuta *startFallAlertCountdown()*. Donde si se acaba la cuenta atrás, se ejecuta *executeEmergencyActions()*, la cual se encarga de hacer la llamada y enviar los SMS con la ubicación.

(RF25): La cuenta atrás de alerta está implementada en la clase *FallDetectionService* con *ALERT_COUNTDOWN_SECONDS = 15*. Durante estos 15 segundos se actualiza la interfaz, la notificación (en el panel de notificaciones del móvil) y el estado interno del servicio.

(RF26): El usuario puede cancelar una falsa alarma antes de que termine la cuenta atrás. Al pulsar el botón, la app envía *ACTION_CANCEL_ALERT* y se ejecuta *cancelFallAlert()*, cancelando el temporizador y las acciones de emergencia.

(RF27): Durante la cuenta atrás, la app activa vibración y sonido en *startAlertAlarm()*. La vibración se gestiona con las librerías *Vibrator/VibrationEffect* y los pitidos repetidos con *ToneGenerator*.

(RF28): La ubicación de emergencia se obtiene en la clase *EmergencyLocationProvider*. Se intenta obtener la ubicación actual y, si no está disponible dentro del tiempo límite, utiliza la última ubicación conocida.

(RF29): Se genera un enlace de Google Maps usando las coordenadas disponibles. Se construye el enlace con el formato <https://maps.google.com/?q=latitud,longitud> y se añade al SMS de emergencia.

(RF30): Si la alerta no se cancela, la app envía un SMS de emergencia a todos los contactos guardados con *sendEmergencySms()*. La función usa la librería *SmsManager* y divide el mensaje en varias partes es muy largo.

(RF31): Si la alerta no se cancela, la app llama al primer contacto de emergencia guardado. Esta acción se realiza desde *callFirstContact()*, usando el primer elemento de la lista de contactos.

(RF32): La llamada de emergencia se hace en modo altavoz al usar *TelecomManager.EXTRA_START_CALL_WITH_SPEAKERPHONE*. Se hace para que el usuario pueda intentar comunicarse aunque el teléfono no esté físicamente cerca.

(RF33): Se gestiona la reconexión automática si el WebSocket falla. *FallDetectionService* ejecuta *scheduleReconnect()*, con tiempo extra progresivo ($n+2$) hasta un máximo de 30 segundos (siempre que no se haya detenido manualmente).

(RF34): Si el Arduino pierde la conexión WIFI en modo normal, se detecta que *WIFI.status()* ya no es *WL_CONNECTED*, detiene mDNS y se reinicia (*ESP.restart()*). Tras reiniciarse, vuelve a intentar conectarse.

(RF35): La app permite compartir/exportar los logs generados con *DataLogger*. Utilizando *FileProvider*, permitiendo enviar los archivos de log a otras aplicaciones. Se hace para no tener que buscarlos dentro del explorador de archivos del teléfono móvil.

Requisitos no funcionales:

(RNF1): La interfaz de la aplicación es bastante sencilla, pero es discutible el decir que no es lo más adecuada para usuarios de alta edad. Parte del diseño está hecho para ayudar mas al desarrollador que al usuario. Dejo la decisión de si se cumple este requisito al lector. (Ver Anexo X de imágenes de la aplicación).

(RNF2): Si nos basamos en los resultados del estudio estadístico, específicamente en el F1 score, se han detectado correctamente el 98,4% de las caídas.

(RNF3): El sistema inicia el flujo de emergencia inmediatamente después de recibir una caída confirmada. Cuando el Arduino envía el mensaje de caída, *FallDetectionService* lo

procesa y ejecuta la cuenta atrás de 15 segundos como medida para permitir cancelar falsas alarmas antes de enviar SMS o realizar llamadas.

(RNF4): El loop de monitorización de ejecuta a 25Hz, haciéndose el muestreo cada ~40ms, lo cual es más que suficiente para que sea considerado “en tiempo real”.

(RNF5): El sistema mantiene la monitorización mientras el Arduino y el teléfono estén encendidos y conectados a la misma red. FallDetectionService (la app) funciona como foreground service que mantiene la conexión activa incluso cuando la aplicación no está en primer plano.

(RNF6): La conexión es robusta porque admite varios mecanismos de conexión. La app puede encontrar automáticamente el dispositivo mediante mDNS/NSD, pero también se puede introducir manualmente un hostname o dirección IP. Si la conexión falla de forma inesperada, el servicio ejecuta una lógica de reconexión progresiva ($n + 2s$).

(RNF7): El algoritmo reduce falsos positivos al no confirmar una caída únicamente por un pico de aceleración. Comprueba fall energy, velocidad angular, postura tumbada e inactividad. Como resultado de los tests, los falsos positivos fueron el 1,4% de los tests.

(RNF8): La app da una oportunidad para cancelar falsas alarmas. Al recibir una caída confirmada, muestra una alerta con una cuenta atrás de 15 segundos y un botón de falsa alarma. Si el usuario pulsa este botón, no se envían SMS ni se realiza la llamada de emergencia.

(RNF9): La comunicación por WIFI está diseñada para funcionar en un entorno doméstico normal, siempre que el Arduino y el teléfono estén conectados a la misma red local y exista cobertura suficiente. El cumplimiento final de este requisito depende de la calidad de la red WIFI y la distancia al router.

(RNF10): Se ha logrado un tamaño bastante reducido al usar una custom PCB, siendo el tamaño del dispositivo 50mm x 60mm x 20mm.

(RNF11): El calculo del tiempo de uso se puede hacer con la siguiente ecuación: $\text{horas} = \text{mAh de la batería} / \text{mA de media}$. Un Arduino usando WIFI de manera constante y alimentando el sensor BNO055 consume aproximadamente 60mA, por lo que el tiempo de uso estimado es de ~8.33 horas. No es lo mas adecuado, pero es facilmente mejorable usando una bateria mas grande. Y al usar el PowerBoost 500C que usa un conector JST, el cambio es muy sencillo.

(RNF12): La app solicita permisos relacionados únicamente con las funciones implementadas: red, SMS, llamadas, notificaciones, ubicación, vibración y foreground service. Estos permisos están justificados por el flujo de emergencia, ya que la app necesita avisar al usuario, obtener ubicación, enviar SMS y llamar a un contacto en caso de caída.

(RNF13): La persistencia de datos está implementada en varias capas. El ESP32 guarda las credenciales WIFI en preferences, la app guarda la configuración del dispositivo mediante DeviceConfigStorage, los contactos mediante EmergencyContactStorage y los logs mediante DataLogger. La información relevante se mantiene tras reinicios de la app o del dispositivo.

(RNF14): El sistema genera logs suficientes para probar y ajustar el algoritmo. La app almacena mensajes WebSocket, datos debug, muestras de ventanas de evento y eventos internos de la aplicación en archivos locales como CSV y JSONL. Esto permite analizar el comportamiento del detector y revisar los casos de caída o falsa alarma posteriormente.

(RNF15): El sistema contempla varios casos de fallo sin bloquear completamente la aplicación. Si faltan permisos, la app puede mostrar avisos y evitar ejecutar acciones no autorizadas. Si no hay contactos, el servicio registra el problema y no intenta enviar avisos. Si no hay ubicación disponible, puede enviar el SMS indicando que la ubicación no está disponible. Y si falla la conexión WebSocket, se intenta reconectar automáticamente.

(RNF16): La app está diseñada para ejecutarse en dispositivos Android compatibles con WIFI, notificaciones, SMS, llamadas, ubicación y vibración. Además, al usar APIs estándar de Android y un minSdk compatible con dispositivos relativamente antiguos, el sistema no depende de hardware especializado más allá de un teléfono móvil relativamente moderno.

(RNF17): El firmware soporta hasta cinco redes WIFI guardadas, y la app permite gestionar varios contactos de emergencia. El envío de SMS se realiza a todos los contactos cargados, mientras que la llamada se dirige al primer contacto configurado.

(RNF18): La app proporciona visibilidad sobre las acciones importantes del sistema. Durante una caída muestra una alerta clara, una cuenta atrás visible y señales mediante sonido y vibración.

(RNF19): El sistema implementa recuperación automática en varios escenarios. Tras un reinicio o corte de alimentación, el ESP32 vuelve a cargar las credenciales guardadas e intenta conectarse a las redes configuradas. Si pierde la conexión WIFI durante el modo normal, se reinicia y vuelve a intentar la conexión. En la app, si el WebSocket falla de forma inesperada, FallDetectionService hace varios intentos de reconexión.